Guía de trabajo Git y GitHub de Latch

Steven Cuadra

Alfredo Montalvan

Caleb Tardencilla

Aris Gavarrete

Facultad de Ingeniería y Arquitectura, Universidad Americana (UAM)

CNU0525: Programación de Aplicaciones de Escritorio, Grupo 2

José A. Durán García

26 de octubre de 2026

Documento auxiliar independiente

Edición editorial corregida

Resumen

Esta guía auxiliar explica el trabajo con el repositorio de Latch para un equipo de cuatro a doce colaboradores. Se subordina al PRD, TRD, Backend Schema, App Flow, Brief UI/UX y Plan de implementación. Conserva sus versiones, identificadores, responsables y calendario. La regla adoptada en ADR-15 es main protegida, ramas cortas y PR con revisión ajena. Las candidatas y hotfix efímeros siguen siendo una enmienda propuesta. La guía relaciona incidencias, ramas, mensajes, revisiones y evidencia; distingue integración técnica de aceptación del entregable. Incluye un kit de texto con formularios, reglas, plantillas, hooks y workflows para revisar antes de su adopción. Los listados extensos están en archivos editables; los apéndices explican su uso. Las comprobaciones documentales y las simulaciones locales no acreditan configuración remota, ejecución de la aplicación, resultados CP ni CI satisfactoria. La adopción se conserva como propuesta fechada desde el 26 de octubre de 2026 y requiere registrar las decisiones y los ensayos que correspondan.

Palabras clave: control de versiones, Git, GitHub, trazabilidad, configuración, Latch

Ruta de consulta

Para iniciar trabajo, consulte la regla vigente de ADR-15, la calidad de entrada de la incidencia y el procedimiento de PR. Antes de configurar GitHub, revise la adopción y los controles del kit. Para recuperación, consulte primero los riesgos de cada comando. Los enlaces subrayados permiten saltar a secciones o abrir archivos auxiliares.

Apéndice A: flujo de ramas, incidencias, CI y versiones.
Apéndice B: registro y revisión de cambios.
Apéndice C: cuentas, etiquetas, milestones y Project.
Apéndice D: CI, publicación y ajustes al POM canónico.
Apéndice E: instalación, mensajes y protección local.
Apéndice F: General y reglas propuestas.
Apéndice G: consulta de términos y recuperación.
Apéndice H: fuentes y matriz de las 96 TAR.

Contenido

Principios y objetivos de la estrategia
Instalación y configuración de Git en Windows 11 x64
Modelo de ramas
Política de integración y conflictos
Convención de commits
Versionado y publicación
Configuración del repositorio GitHub
Issues como registro operativo
    Identificadores, formularios y calidad de entrada
    Taxonomía de etiquetas y contraste
    Milestones y fechas
    GitHub Projects v2
    Enlaces entre issue, rama, PR y commit
    Ciclos de vida y transiciones
    Comentarios, triage y obsolescencia
Pull requests y revisión cruzada
CI y publicación con GitHub Actions
Seguridad, dependencias y cumplimiento
Escalabilidad y mantenimiento
Recuperación y operación diaria
Integración con el Plan de implementación
Adopción en el calendario real
Riesgos del flujo y contingencias
Lista de verificación de entrega
Referencias

Principios y objetivos de la estrategia

La estrategia propone una sola línea permanente de integración en main, trabajo breve vinculado a una incidencia y revisión independiente antes de integrar. Se aplicará a Latch, aplicación de escritorio Java 21, JavaFX 21.0.13, PostgreSQL 18.6, Hibernate 7.4.6.Final y JasperReports 7.0.8, construida con Maven 3.9.16 mediante Wrapper y el objetivo único verify. Estas versiones se conservan como base del encargo; este documento no acredita su resolución ni ejecución. El flujo cubre de cuatro a doce colaboradores sin agregar ramas permanentes por persona o módulo.

La guía identifica como destino previsto https://github.com/LitSoloSteven/Latch. Esta edición revisó el repositorio local en modo de lectura; su archivo .git/config no contiene un remoto configurado. La visibilidad pública, el tipo de cuenta y main predeterminada se mantienen como condiciones de la propuesta que Steven debe confirmar al adoptarla. No se revalidó la consulta histórica de metadatos descrita en la edición anterior ni se deducen de ella protecciones, accesos o resultados de CI.

La línea base obligatoria proviene de los seis documentos oficiales. Esta guía es auxiliar y no agrega entregables de producto. Las reglas operativas adicionales y los archivos del kit son propuestas hasta que el equipo registre su adopción por D44; las metas de tamaño, plazos internos y alias son recomendaciones salvo acuerdo expreso. Steven Cuadra coordina la adopción y conserva seguridad y concurrencia; Alfredo Montalvan responde por datos y persistencia; Caleb Tardencilla por FXML, temas y pruebas; Aris Gavarrete por HTTP, reportes y distribución. Las responsabilidades y relevos proceden del Plan (Cuadra et al., 2026f). La configuración descrita supone GitHub Free para un repositorio personal público, sin beneficios Education; el plan contratado debe confirmarse (GitHub, s. f.-m).

La Tabla 1 fija metas de operación propuestas. Steven las calcula diariamente a partir de eventos fechados de Issues, PR y Project. En el cierre de cada período registra mediana,

percentil 85 cuando exista una muestra suficiente y número de observaciones. Sin eventos, el resultado es Pendiente o Sin datos; nunca cero horas por ausencia de registro.

Tabla 1

Criterios Medibles de Operación

Indicador y meta	Quién y cuándo	Cálculo y comprobación
PR listo sin respuesta ≤24 h; PR abierto >48 h activa triage	Revisor responde; Steven revisa a diario	Tiempo calendario desde ready_for_review hasta primera revisión; contar PR de más de 48 h, incluso drafts, identificándolos.
Rama corta ≤5 días calendario; cero ramas excedidas sin resolución	Autor divide trabajo; Aris revisa al cierre diario	Desde primer push remoto registrado hasta eliminación; si se desconoce creación, usar primer evento disponible y rotular proxy.
Ciclo TAR a Terminada: meta interna mediana ≤5 días	Steven al cerrar período	Desde primer paso a En curso hasta aceptación en Terminada; incluir bloqueo en tiempo total y reportarlo por separado.
Una TAR En curso por persona; PR recomendado ≤400 líneas netas y ≤15 archivos	Autor al iniciar; revisor al abrir PR	Diff excluye generado; dividir por criterio verificable si excede. Excepción D44 con motivo.
100 % de integraciones con revisor ajeno y checks requeridos	Quien integra, en cada PR	Revisión aprobada vigente, conversaciones resueltas y checks ci/policy del commit integrado.
100 % de cierres con RF/RN o decisión, TAR/DEF, CP/RA y evidencia	Persona A, antes de cerrar	Enlace y SHA revisables; fecha y entorno; exclusiones justificadas; D47.

Nota. Metas operativas propuestas; su adopción se registra por D44. Los indicadores requieren eventos reales.

Main siempre desplegable es una obligación de conservación: cada integración debe permitir construir el paquete y mantener las funciones aceptadas. Si una comprobación posterior falla, Steven detiene nuevas integraciones funcionales, abre DEF y decide revertir o

corregir con prioridad. La frase no describe el estado de ejecución del esqueleto recibido. No se utilizan porcentajes de avance derivados de columnas sin aceptación.

Instalación y configuración de Git en Windows 11 x64

Cada integrante instala Git for Windows 2.51.0.windows.1 desde el archivo oficial enlazado por git-scm.com, selecciona x64, verifica firma del instalador y registra su huella y origen. La versión es una fijación del encargo, no una afirmación de versión más reciente ni de descarga realizada. Un parche de seguridad se tramita por D44. Después se comprueban git --version y where.exe git para identificar el ejecutable que usa IntelliJ y la terminal (Chacon & Straub, 2014, sección 1.5).

Se eligen OpenSSH y Git Credential Manager durante la instalación. La autenticación HTTPS se conserva en el almacén seguro de Windows; se prohíbe credential.helper store. Cada persona usa identidad y correo verificado propios. El nombre no sustituye la autorización de GitHub. La configuración global vive en la cuenta Windows; la local en .git/config prevalece para este clon. Steven verifica la instalación en una sesión compartida sin solicitar claves privadas ni contraseñas (Chacon & Straub, 2014, sección 8.1).

El archivo docs/instalacion-git.md del Apéndice E contiene los comandos completos. La Tabla 2 resume los valores obligatorios. Los alias son ayudas de lectura y no cambian la política de integración. Cada autor conserva git config --show-origin --list con los valores sensibles redactados en una evidencia de configuración.

Tabla 2

Configuración de Referencia de Git

Valor	Ámbito y responsable	Verificación
user.name y user.email propios	Global; cada integrante	git config --show-origin --get user.email; correo verificado o noreply de GitHub.

Valor	Ámbito y responsable	Verificación
core.autocrlf=false; core.eol=lf	Local; cada clon	git config --local --get core.autocrlf; .gitattributes aplica excepciones cmd/bat.
credential.helper=manager	Global; cada integrante	git credential-manager --version y acceso por navegador; no registrar tokens.
init.defaultBranch=main; fetch.prune=true	Global; cada integrante	Comprobar configuración y referencias remotas tras fetch.
pull.ff=only; core.hooksPath=.githooks	Local; cada clon	Instalar scripts/install-hooks.ps1; intento válido y rechazo de mensaje inválido.
gpg.format=ssh; commit.gpgsign=true; tag.gpgsign=true	Global o local si se usa clave dedicada	git log --show-signature y Verified en GitHub; comprobar allowed_signers fuera del repo.

Nota. Configuración propuesta por clon y por cuenta. No se aplicó durante la corrección documental.

La firma SSH propuesta usa una clave Ed25519 con passphrase, fuera del repositorio, cuya parte pública se registra en GitHub como Signing Key. Se configura user.signingkey con la ruta pública correcta. Para verificar localmente se mantiene allowed_signers fuera del repositorio; la confianza de ese archivo debe ser revisada por su propietario. La alternativa GPG exige seleccionar openpgp, el ID de clave y publicar únicamente su clave pública. Cada integrante prueba un commit firmado antes de la primera PR; -S por sí solo no acredita que GitHub lo considere Verified (GitHub, s. f.-w).

La adopción de .gitattributes requiere una PR de normalización separada: autor ejecuta git add --renormalize ., revisa el diff con el revisor y repite verify. Caleb reserva la ventana de FXML y Alfredo la de SQL para evitar conflictos de finales de línea mientras otros editan. No se cambian configuraciones globales desde un script de clonación; install-hooks.ps1 solo configura este repositorio.

Modelo de ramas

Se propone una adaptación de GitHub Flow con main y ramas de trabajo cortas, más una candidata efímera cuando el calendario lo requiera. GitHub Flow describe creación de rama, commits y revisión mediante PR (GitHub, s. f.-l). El modelo original de Driessen mantiene master y develop como líneas permanentes y separa release y hotfix (Driessen, 2010). Latch conserva una sola línea permanente porque su calendario reúne seis períodos y no compromete soporte simultáneo de varias versiones. La candidata permite revisar una entrega académica identificada sin mantener develop.

La comparación con trunk-based se formula como decisión local: se toman integración frecuente y lotes pequeños, pero se conservan ramas y PR con aprobación ajena. No se afirma aplicar de manera estricta una metodología trunk-based ni se atribuye esa valoración a una fuente que no la desarrolla. Al crecer a doce personas, se incrementa la capacidad de revisión por paquetes y se limita el trabajo en curso; el número de líneas permanentes permanece en uno.

La Tabla 3 determina creación, vencimiento y eliminación. Los plazos se cuentan en horas o días calendario desde el primer push registrado. Antes de vencer, el autor integra una parte aceptable, divide la TAR o registra bloqueo; una excepción no prorroga automáticamente la rama. Aris conserva el SHA y revisa ramas vencidas con Steven diariamente. Ninguna rama se crea para representar un estado del tablero.

Tabla 3

Ramas, Responsables y Vida Máxima

Rama y origen	Creación y vida máxima	Eliminación y control
main; raíz aceptada del repositorio	Steven crea una vez; permanente	Nadie la elimina; ruleset bloquea borrado y force push.
feature/TAR-nnn-descripcion-corta; desde main	Autor de TAR preparada; 5 días	Autor o Aris tras squash y evidencia de PR; limpieza automática verificada.

Rama y origen	Creación y vida máxima	Eliminación y control
fix/DEF-nnn-descripcion-corta; desde main si corrige la línea vigente; desde release/* si corrige esa candidata	Responsable del defecto tras triage; 5 días	PR a la base correspondiente: squash hacia main o merge hacia release. Conservar DEF/CP y trasladar la corrección al consolidar.
docs/TAR-nnn-descripcion-corta; desde main	Autor documental; 3 días	Autor tras revisión de fuentes y PR integrada.
build/TAR-nnn-descripcion-corta; desde main	Aris o autor asignado; 3 días	Aris tras verify y PR integrada.
release/vX.Y.Z o release/vX.Y.Z-rc.n; desde main	Aris con autorización registrada de Steven; 3 días	Aris tras consolidar en main, etiquetar y conservar SHA; excepción de eliminación descrita.
hotfix/DEF-nnn-descripcion-corta; desde tag aceptado	Autor asignado por Steven; 48 h	Aris tras PR a main y nueva etiqueta; traslado a candidata si sigue abierta.

Nota. La candidata y el hotfix requieren adopción de la enmienda a ADR-15. La regla vigente conserva main y ramas cortas revisadas.

Las descripciones usan minúsculas ASCII y números separados por guion; no llevan espacios, tildes ni nombre de persona. El validador rechaza ramas ajenas al patrón del Apéndice E. Se recomienda hasta 80 caracteres; los identificadores son exactamente de tres dígitos para TAR y DEF, sin convertir el número de issue en identificador del Plan. La expresión de release permite candidata porque el Plan fija v0.9.0-rc.1; el nombre vX.Y.Z designa el caso sin sufijo.

Patrón de nombres de rama

`config/branch-regex.txt` — Patrón íntegro; la sintaxis se comprueba en el kit.

ADR-15 del TRD ya adopta main protegida, ramas cortas y PR revisada, y descarta Git Flow con develop permanente. La desviación propuesta se limita a permitir candidatas y hotfix efímeros y a precisar su método de integración. No se atribuye una decisión contraria al documento de origen (Cuadra et al., 2026d). El archivo docs/adr/ADR-15-enmienda-git.md conserva el identificador y registra estado Propuesta, motivo, impacto y Steven como

responsable. Caleb revisa la coherencia y Aris la operación; la aceptación del equipo se registra antes de activar reglas, conforme a D44.

Política de integración y conflictos

El autor actualiza su rama con rebase local antes de abrir una PR lista. Primero obtiene origin/main y comprueba un árbol limpio. Git rebase reaplica cambios y genera revisiones nuevas; por ello se limita a la historia propia y se aborta si no se entiende un conflicto (Git, s. f.-a). Si ya publicó su rama, avisa al revisor y usa --force-with-lease únicamente allí. Quedan prohibidos --force, force push en main o release y rebase de historia compartida sin acuerdo explícito de sus autores.

Las ramas cortas se integran en main por squash and merge. Antes de integrar, se valida la cabecera y el cuerpo reales del mensaje preparado, incluidos Refs y el módulo. La persona que integra copia el mensaje validado al diálogo de squash y comprueba que no se haya alterado. Los commits intermedios también pasan por policy; solo los merge commits estructurales de una candidata quedan excluidos del patrón de cabecera. Una corrección dirigida a candidata nace de la revisión vigente de release/* y vuelve a esa misma rama mediante PR y merge commit. No se actualiza esa rama de corrección con main.

La enmienda propone congelar las funciones de main durante la vida de una candidata, limitada a tres días. Aris registra el SHA de origen y consolida la candidata mediante PR por squash hacia main antes de etiquetar. Si main avanzó, el revisor compara ambos árboles y la lista de cambios; esto comprueba una ambigüedad de base, sin presumir que ocurrió una contaminación. Las funciones posteriores ajenas a la candidata deben diferirse o revertirse mediante otra PR revisada antes de consolidar. Si el equipo decide incluirlas, registra D44 y abre un nuevo ciclo de candidata con su aceptación completa. No se recrea automáticamente una candidata desde una main que contenga funciones excluidas. GitHub no admite merge commits en una rama con historial lineal: General permite squash y merge, el ruleset de main

restringe a squash e historial lineal y el de release permite merge sin exigir linealidad (GitHub, s. f.-g).

Si el conflicto afecta FXML, Caleb registra en el issue archivo, titular temporal, revisión de partida y vencimiento de la reserva. Autor y revisor reconstruyen estructura, fx:id, onAction y recursos, abren la pantalla en Claro/Oscuro y conservan capturas. No se acepta una resolución que solo quite marcadores. Si Caleb es autor, Steven realiza la revisión; Aris cubre el recorrido de interfaz cuando se activa relevo. La reserva termina al integrar o al transferirse por comentario fechado.

Si el conflicto afecta migraciones, Alfredo coordina la secuencia y verifica dependencias. V001 a V004 aplicadas o publicadas son inmutables; cualquier modificación futura del esquema requiere la decisión y la numeración que correspondan en D44. Dos ramas no editan el mismo archivo reservado. T001 es el fixture sintético de pruebas y R001 contiene las consultas de los diez reportes y dos complementos; ninguno es un script de restauración. Los oráculos de reportes se relacionan con CP-067–CP-076 y las exportaciones aplicables con CP-086 y CP-100. La restauración aislada se comprueba con CP-095 y el respaldo frente a escritura LAN con CP-051, usando el procedimiento oficial y los datos sintéticos pertinentes. Una reversión Git no revierte automáticamente la base de datos (Cuadra et al., 2026e, pp. 114–116; Cuadra et al., 2026f, catálogo CP).

Convención de commits

La adaptación conserva los tipos fijados feat, fix, test, docs y build. Cada commit representa un cambio revisable, con descripción en español y referencia TAR o DEF en la cabecera. El alcance M01 a M12 es opcional en la cabecera para mantener válida la convención previa; si falta, se exige el pie Module. En ambos casos se exige Refs con el mismo identificador. Conventional Commits define tipo, alcance opcional, descripción y pies; Latch añade estas restricciones de trazabilidad como decisión propia (Conventional Commits, s. f.).

Se admite feat(M06): registrar pago parcial (TAR-038), seguido de una línea vacía y Refs: TAR-038. Para el ajuste de Wrapper, la tarea canónica es TAR-004: build: fijar Wrapper (TAR-004), con Refs: TAR-004 y Module: M12. Un DEF puede justificar un commit fix; si consume una TAR existente se documentan ambos sin cambiar sus identificadores. Un cambio incompatible lleva ! y un pie BREAKING CHANGE que identifica el contrato afectado. El autor prepara los mensajes de varias líneas en un archivo UTF-8.

La Tabla 4 permite comprobar la convención sin depender de la interpretación del revisor. El hook commit-msg y el script de policy comparten la misma validación; las pruebas locales de mensajes no acreditan una ejecución remota de GitHub Actions. Los merge commits estructurales de release se excluyen del patrón de cabecera; sus commits de producto siguen sujetos al patrón.

Tabla 4

Ejemplos de Mensajes y Resultado Esperado

Mensaje o situación	Resultado	Motivo
feat(M06): registrar pago parcial (TAR-038) + Refs: TAR-038	Válido	Tipo, módulo y referencia coinciden.
fix(M01): impedir acceso reservado (DEF-001) + Refs: DEF-001	Válido como ejemplo	DEF-001 es ilustrativo; asignarlo al triage antes de usarlo.
docs: registrar revisión (TAR-016) + Refs: TAR-016 + Module: M12	Válido	Conserva cabecera previa; módulo explícito en pie.
feat: mejorar todo	Inválido	Falta identificador y módulo; descripción sin límite verificable.
chore(M01): limpiar (TAR-001)	Inválido	Tipo fuera de la convención del proyecto.
feat(M13): guardar (TAR-017)	Inválido	Solo M01 a M12.
fix(M06): corregir pago (DEF-001) + Refs: TAR-038	Inválido	La referencia del pie no coincide.

Mensaje o situación	Resultado	Motivo
Commit con Closes #n	Inválido	Palabras de cierre se revisan en PR, donde se verifica aceptación.

Nota. Los mensajes muestran líneas separadas por «+» solo para abreviar la tabla; los pies reales van después de una línea vacía.

Cada integrante instala los hooks en su clon. pre-push bloquea las actualizaciones directas de main y release/*; la creación y la retirada de candidatas siguen el procedimiento autorizado descrito en el Apéndice E. El hook ejecuta verify antes de publicar cambios de trabajo. Los hooks son controles locales y las reglas de GitHub son el control compartido. Las PR de Dependabot se trasladan a una rama build con TAR asignada, mensaje válido y verificación; el bot no recibe una excepción de revisión.

Versionado y publicación

La versión usa SemVer: MAJOR identifica cambio incompatible del contrato declarado, MINOR capacidad compatible y PATCH corrección compatible. Antes de 1.0.0 el contrato está en desarrollo, pero cada entrega sigue siendo identificable e inmutable. El contrato de Latch abarca esquema compatible, formato de respaldo y exportación y comportamientos comprometidos por RF/RN; Aris describe incompatibilidades y Alfredo las migraciones. La candidata rc precede a la versión final correspondiente; la sucesión v0.9.0-rc.1 a v1.0.0 sigue el Plan, sin afirmar que sean la misma versión (Preston-Werner, s. f.).

La Tabla 5 conserva las etiquetas y sus ventanas. El Plan programa consolidar v1.0.0 el 30 de noviembre y mostrarla en HIT-10 el 1 de diciembre; se distinguen ambos momentos. La publicación puede quedar bloqueada si no satisface aceptación. No se modifica una etiqueta publicada para aparentar cumplimiento de una fecha (Cuadra et al., 2026f).

Tabla 5

Etiquetas y Entregas del Plan

Etiqueta	Fecha prevista	Criterio y responsable
v0.1.0	27/10/2026	HIT-01; fundamentos, dos temas y concurrencia. Aris prepara; Steven verifica.
v0.2.0	03/11/2026	HIT-02; dominio, ORM y contrato sin solape. Alfredo verifica datos.
v0.3.0	10/11/2026	HIT-04; pago, anulación y garantía íntegros. Alfredo y Steven verifican.
v0.4.0	17/11/2026	HIT-05; diez reportes y respaldo restaurable. Aris y Alfredo verifican.
v0.9.0-rc.1	25/11/2026	HIT-07; congelación y regresión Must. Caleb coordina CP.
v1.0.0	30/11/2026; cierre 01/12/2026	TAR-092 consolida; HIT-10 exige RA1 a RA5 y archivos íntegros.

Nota. Fechas y etiquetas del Plan. Consolidación el 30/11 y demostración HIT-10 el 01/12 son momentos distintos.

Steven crea una etiqueta anotada y firmada desde una revisión aceptada de main, a solicitud de Aris. El ruleset de tags separa creación de inmutabilidad: la excepción para crear no habilita actualizar ni borrar. La release automatizada repite CI, construye en Windows, genera app-image con jpackage, comprime ZIP y calcula SHA-256. El manifiesto contiene versión, SHA Git, fecha UTC, entorno observado, versiones declaradas, compatibilidad V001 a V004, nombre del ZIP y huella. Aris compara la huella tras descargar y extraer el archivo; SHA-256 demuestra integridad respecto del manifiesto, no ejecución ni firma del autor.

GitHub puede generar notas con PR integradas y colaboradores; Aris las revisa, incorpora requisitos, CP, migraciones, riesgos residuales y pendientes antes de publicar (GitHub, s. f.-e). El workflow crea borrador y falla si ya existe una release para evitar

reemplazar artefactos. Publicar el borrador exige registro de aceptación de Steven. El ZIP se adjunta a la release, no al historial del código.

El paso de candidata a v1.0.0 exige cien CP y variantes aplicables con resultado y entorno, cero defectos Crítica o Alta que afecten Must, aceptación de requisitos obligatorios, evidencias RA1 a RA5 y recorrido del paquete en Windows 11 x64 limpio. También requiere migraciones desde cero, restauración aislada, manual y seis documentos, inventario de licencias completo y dos copias verificadas. Caleb consolida resultados; Alfredo verifica datos; Aris comprueba paquete; Steven acepta el cierre D47. Una excepción queda identificada y aceptada expresamente; no vuelve opcional un Must incumplido.

Configuración del repositorio GitHub

La propuesta se limita al repositorio personal LitSoloSteven/Latch, público y con main predeterminada. Solo se publican código, documentación revisada y datos sintéticos. Steven completa About con la descripción y topics del Apéndice F, sin agregar una web inexistente. Issues y Projects permanecen habilitados. La visibilidad pública no permite publicar datos operativos ni usar cuentas compartidas. Cada integrante activa autenticación de dos factores en su cuenta; el propietario revisa accesos al cerrar un sprint.

GitHub distingue propietario y colaboradores en un repositorio personal. No se asignarán Maintain ni Triage como permisos técnicos separados. Steven administra la cuenta propietaria; Aris desempeña mantenimiento por procedimiento, con el mismo acceso de colaborador de Alfredo y Caleb. No se comparte la contraseña del propietario para ejecutar el relevo (GitHub, s. f.-q). La Tabla 6 mantiene esta distinción para que el RACI no se confunda con privilegios del servicio.

Tabla 6

Permisos y Responsabilidad en el Repositorio Personal

Persona o función	Permiso aplicable	Límite y comprobación
Steven Cuadra, LitSoloSteven	Propietario; capacidad de administración	Configura reglas, accesos y seguridad; no omite revisión. Revisar lista de colaboradores y reglas efectivas.
Aris Gavarrete, Arisgav	Colaborador con escritura	Mantiene builds, PR y borradores. Función Maintain operativa; no privilegio independiente.
Alfredo Montalvan, wetalff	Colaborador con escritura	Datos y persistencia; integra solo mediante PR protegida y revisión ajena.
Caleb Tardencilla, CTardencilla	Colaborador con escritura	FXML y pruebas; triage operativo sin un rol Triage separado.
Nuevos colaboradores, hasta doce	Colaborador con escritura tras asignación de módulo	Steven verifica necesidad, aceptación de invitación y revisión cruzada. Observación pública no requiere Write.

Nota. Cuentas indicadas en el documento de origen. Los accesos efectivos e invitaciones deben comprobarse al adoptar. Maintain y Triage describen funciones operativas; no son roles técnicos separados de este repositorio personal.

En Settings > General se habilitan squash y merge commit y se desactiva rebase merge. El ruleset de main conserva solo squash y el de release permite merge. La propuesta usa el título de la PR como título inicial del squash y el cuerpo como base, pero el mensaje final se prepara en el bloque «Squash message» de la plantilla y se copia completo al diálogo de integración. policy valida ese bloque real y su coincidencia con el título; no fabrica pies ni un módulo. La edición posterior del diálogo exige comprobación humana. Se propone desactivar auto-merge para las PR que requieren esa preparación manual; habilitarlo exige una decisión que garantice el mismo mensaje final. La eliminación automática solo se usa para ramas de

trabajo; la retirada de release sigue su procedimiento. GitHub documenta la disponibilidad de auto-merge para Free público, sin que ello obligue a adoptarlo (GitHub, s. f.-p).

Steven importa main.json con PR obligatoria, una aprobación ajena, Code Owner, descarte de aprobaciones obsoletas, aprobación del último push por otra persona, conversaciones resueltas, checks ci y policy estrictos, historial lineal y bloqueo de force push y eliminación. No hay bypass de revisión en main. Release conserva PR y checks, exige método merge y bloquea force push; su eliminación posterior se permite por decisión explícita para conservar su carácter efímero. Aris es responsable procedimental de borrarla tras conservar SHA y etiqueta. Si se quiere bloqueo técnico durante la candidata, Steven agrega temporalmente deletion y lo retira con D44 antes de limpiar.

Los rulesets de ramas y tags están disponibles en repositorios públicos Free; evaluate no se toma como capacidad de esta base. Se usa active después del ensayo y disabled durante preparación controlada. La propuesta no depende de push rulesets ni reglas de metadatos que no estén incluidas en Free. No se exige merge queue: el flujo usa checks estrictos y una integración a la vez coordinada por el revisor (GitHub, s. f.-c; GitHub, s. f.-s).

CODEOWNERS está en la raíz, con las cuentas reales y patrones por paquetes propuestos. GitHub exige permisos de escritura a los propietarios de código; aplica la primera ubicación encontrada y la última coincidencia del archivo (GitHub, s. f.-a). Steven verifica que no haya una copia diferente en .github o docs y que los paquetes coincidan con el código antes de exigir su aprobación. Los patrones propuestos no acreditan que esos paquetes ya existan. Dos propietarios por área permiten relevo; si uno es autor, aprueba el otro. La persona A del RACI sigue respondiendo por aceptación aunque no sea quien revisa el código.

La Tabla 7 distingue las capacidades documentadas del servicio de las condiciones de adopción. Esta corrección no consultó el estado actual del remoto ni verificó la suscripción, las invitaciones, las protecciones o las ejecuciones de workflows.

Tabla 7

Funciones de GitHub y Condiciones de Uso

Función	Condición documentada	Aplicación propuesta
PR, Issues y repositorio público	Base Free público; sin Education requerido	Uso de equipo con cuentas individuales; verificar acceso antes del primer trabajo.
Rulesets de ramas/tags y CODEOWNERS	Disponibles en Free público	Importar y probar; no afirmar activación por tener JSON.
Auto-merge	Disponible en Free público	Disponible; desactivado en el procedimiento manual de squash hasta acordar un mecanismo equivalente.
Projects v2 y API	Project de la cuenta; API exige permisos propios	Token project separado, con vencimiento; GITHUB_TOKEN no basta por sí solo.
Secret scanning y push protection	Scanning público gratuito; protección depende de patrones y configuración	Verificar controles disponibles en Security y conservar captura de activación.
Roles Maintain y Triage separados	No asignables aquí como privilegios personales independientes	Distribuir tareas operativas; conservar propietario y colaboradores.
Reglas de push o funciones de pago	Fuera de la base declarada	No usarlas como requisito del kit; cualquier ampliación necesita D44.

Nota. Decisión propuesta. La aceptación se registra con fecha, revisión y entorno.

Issues como registro operativo

Identificadores, formularios y calidad de entrada

Cada incidencia tiene un número GitHub #n y un identificador del proyecto independiente. Steven verifica al triage que TAR-001 a TAR-096 provengan del backlog, CP-001 a CP-100 del catálogo, RSK-01 a RSK-18 del registro y ADR-01 a ADR-22 de decisiones existentes. DEF se asigna correlativamente sin reutilizar números. Una variante de CP conserva CP y describe la variante. Un riesgo o una ADR nuevos fuera del catálogo se proponen por D44 antes de ampliar el registro; los formularios no otorgan esa aprobación.

Los siete formularios completos del Apéndice B crean Tarea TAR, Defecto DEF, Cambio de alcance, Riesgo, Decisión/ADR, Caso de prueba CP y Documentación. Todos requieren identificador, vínculo RF/RN o decisión de gestión, módulo, horas estimadas, precedencia, período, responsable y revisor, aceptación y evidencia sin secretos. El campo evidencia acepta Pendiente con procedimiento antes de ejecutar; no exige inventar un resultado para poder abrir el issue. Los formularios se conservan como YAML bajo .github/ISSUE_TEMPLATE y sus respuestas se convierten a Markdown según GitHub (GitHub, s. f.-x).

La Tabla 8 indica la información que Steven o el dueño de área comprueba antes de Preparada. Los campos obligatorios del formulario no demuestran que un RF exista ni que una estimación sea suficiente. Una incidencia incompleta se registra abierta y se deja fuera del Project, o con Status sin asignar si ya se añadió, con comentario de los datos faltantes y responsable de aclararlos. No recibe estado:Preparada ni se asigna a ejecución. Cuando el triage confirma la información, ingresa a Preparada. Status sin asignar no es un séptimo estado ni una nueva columna; Bloqueada se reserva para una ejecución iniciada que no puede continuar.

Tabla 8

Tipos de Issue y Revisión de Entrada

Tipo	Contenido específico obligatorio	Quién y control de Preparada
Tarea TAR	Trabajo acotado, requisito, horas y precedencia exacta; CP/RA	Responsable estima; Steven compara con backlog; no copiar horas estimadas a reales.
Defecto DEF	Entorno y SHA, pasos, esperado/observado, severidad y evidencia	Caleb reproduce o rotula Pendiente; dueño técnico confirma impacto y vínculo a CP.

Tipo	Contenido específico obligatorio	Quién y control de Preparada
Cambio de alcance	Motivo, alternativa, impacto en horas/plazo/RA y resolución D44	Steven registra decisión y consulta A del área; sin aprobación no cambia la línea base.
Riesgo RSK	P e I de 1 a 5, disparador, mitigación, contingencia y revisión	Dueño del RSK actualiza; Steven compara con registro canónico.
Decisión ADR	Contexto, alternativas, consecuencias, estado y decisión sustituida	Steven mantiene identificador; persona técnica explica motivo e impacto.
Caso CP	Fixture, procedimiento, resultado esperado y resultado observado o Pendiente	Caleb verifica CP existente, variante y trazabilidad; ejecutor se identifica al probar.
Documentación	Archivo/sección, motivo, fuente primaria y verificación documental	Autor y revisor comprueban citas, identificadores y límites de evidencia.

Nota. Antes de Preparada se verifica la información; la falta de datos no es un bloqueo de ejecución.

Defecto Crítica significa pérdida o corrupción financiera, acceso indebido grave o exposición de un secreto activo que impida una entrega segura. Alta impide un recorrido Must sin alternativa aceptable. Media afecta un comportamiento acotado con alternativa y sin comprometer invariantes; Baja describe presentación o documentación sin bloqueo funcional. Son criterios internos propuestos: Caleb clasifica con el dueño técnico, y Steven resuelve desacuerdo antes de priorizar. Must/Should/Could indica prioridad de requisito, mientras severidad indica impacto del defecto; un defecto de apariencia no hereda Crítica por estar en un módulo Must.

Taxonomía de etiquetas y contraste

El archivo config/labels.json define 47 etiquetas, con nombre, color hexadecimal y descripción. El script bootstrap usa gh label create --force de manera reproducible. Los prefijos

tipo:, módulo:, prioridad:, severidad:, estado:, sprint:, ra: y riesgo: evitan mezclar dimensiones. Cada issue lleva un tipo principal, módulo principal, prioridad y período; los DEF llevan severidad y los riesgos el nivel de P × I. RA admite varias etiquetas. El responsable sincroniza estado: con Status al cambiar de columna; Caleb revisa diferencias al triage diario. No hay sincronización automática de todos los campos por el solo hecho de usar un formulario.

La Tabla 9 agrupa los colores usados. Se calcula luminancia sRGB y razón (L mayor + 0,05)/(L menor + 0,05) contra texto negro #000000. Las etiquetas claras se eligen para esa presentación; GitHub controla el render final, por lo que Caleb comprueba ambos temas del sitio tras crearlas. Si el texto efectivo cambia, se recalcula con su color real. Los nombres escritos preservan el significado sin depender del color. El umbral 4,5:1 se adopta del encargo; el cálculo comprueba la paleta del kit, no una captura del sitio.

Tabla 9

Colores y Contraste Calculado de Etiquetas

Prefijo	Fondo	Contraste	Uso
tipo:	#DCEBFA	17.31:1	7 etiquetas; texto #000000
módulo:	#E2E8F0	17.03:1	12 etiquetas; texto #000000
prioridad:	#FFF1B8	18.53:1	3 etiquetas; texto #000000
severidad:	#FFDADB	16.30:1	4 etiquetas; texto #000000
estado:, cierre:	#D8F3E5	17.87:1	7 etiquetas; texto #000000
sprint:	#EDE2F7	16.82:1	6 etiquetas; texto #000000
ra:	#D8F0F5	17.70:1	5 etiquetas; texto #000000
riesgo:	#FFE6CC	17.44:1	3 etiquetas; texto #000000

Nota. Razones sRGB frente a texto negro. El renderizado real de las etiquetas en GitHub debe comprobarse al adoptar.

Los seis estados son Preparada, En curso, En revisión, En prueba, Terminada y Bloqueada. Sprint usa 0, 1, 2, 3, 4 y Cierre. La escala oficial del Plan clasifica P × I como verde

o bajo hasta 7, ámbar o medio de 8 a 14 y rojo o alto desde 15. Con P e I enteros de 1 a 5, el mínimo es 1 y el máximo 25; algunos enteros no son productos posibles, pero eso no cambia los intervalos oficiales. Por ejemplo, 2 × 3 = 6 es verde o bajo, 2 × 4 = 8 es ámbar o medio y 3 × 5 = 15 es rojo o alto. Las etiquetas incluyen el nivel escrito, por lo que el significado no depende del color. Solo la persona A aplica cierre:con-evidencia tras aceptar el resultado.

Milestones y fechas

Aris crea seis milestones de período con las fechas de la Tabla 10 y diez milestones HIT con los compromisos de la Tabla 11. GitHub permite un solo milestone por issue: una TAR conserva el de su sprint; un issue de seguimiento del hito usa HIT-nn y enlaza las TAR, PR y evidencias que lo satisfacen. No se intenta asignar ambos a la misma incidencia. Sprint y RA también figuran en campos de Project para filtrar sin alterar el milestone.

Tabla 10

Milestones por Período

Milestone	Inicio	Vencimiento
Sprint 0	22/10/2026	27/10/2026
Sprint 1	28/10/2026	03/11/2026
Sprint 2	04/11/2026	10/11/2026
Sprint 3	11/11/2026	17/11/2026
Sprint 4	18/11/2026	25/11/2026
Cierre	26/11/2026	01/12/2026

Nota. Fechas del Plan, Tabla 8. Se usa fin del día en America/Managua; no se atribuye confirmación institucional.

Tabla 11

Milestones de Hitos de Entrega

Hito y fecha	Compromiso	Evidencia esperada
HIT-01; 27/10/2026	v0.1.0; FXML y concurrencia.	Video interno corto, consultas y log H6.
HIT-02; 03/11/2026	v0.2.0; dominio con ORM.	SQL y pruebas de repositorios sobre PostgreSQL.
HIT-03; 06/11/2026	Revisión de laboratorio ORM.	Árbol de dependencias y registro de revisión.
HIT-04; 10/11/2026	v0.3.0; circuito financiero.	CP-026 a CP-036 y CP-041 a CP-052.
HIT-05; 17/11/2026	v0.4.0; diez reportes.	PDF, XLSX y oráculos CP-067 a CP-076.
HIT-06; 21/11/2026	Adaptador HTTP.	Registro del servidor S1 y casos HTTP.
HIT-07; 25/11/2026	v0.9.0-rc.1 congelada.	Informe de pruebas, app-image ZIP y manifiesto.
HIT-08; 26/11/2026	Entrega interna y sesión.	Manifiesto, hashes y protocolo de apertura.
HIT-09; 30/11/2026	Ensayo final.	Registro de tiempos por bloque y regresión dirigida.
HIT-10; 01/12/2026	v1.0.0 y cierre.	Etiqueta, historial de defectos y paquete archivado.

Nota. Plan, Tabla 10. Aris administra milestones; la persona A acepta el hito con evidencia. Los títulos HIT no sustituyen etiquetas SemVer.

GitHub Projects v2

Se propone un Project de la cuenta LitSoloSteven. El script create-project.ps1 crea el proyecto y sus campos; la URL y los IDs se guardan después de ejecutarlo. La Tabla 12 fija el esquema. Los campos de selección permiten filtrar por opción (GitHub, s. f.-d). Quien hace triage transcribe los datos del formulario a los campos y Caleb comprueba consistencia. Horas

reales queda vacío hasta que exista registro de trabajo, aun cuando Horas estimadas tenga valor.

Tabla 12

Campos del Project y Reglas de Registro

Campo y tipo	Valor permitido	Responsable y momento
Status, selección	Seis estados del tablero	Responsable al cambiar estado; A valida Terminada.
TAR, texto	TAR-nnn; DEF/CP/RSK/ADR cuando corresponda	Steven al triage; comparar con catálogo, no con #n.
Módulo, selección	M01 a M12 o Transversal	Dueño técnico; principal en campo, secundarios en cuerpo.
Horas estimadas y Horas reales, número	Horas decimales; reales vacío hasta registro	Autor estima al preparar y registra reales por jornada.
Sprint, selección	Sprint 0 a 4 y Cierre	Steven al planificar; cambio con motivo.
RA, texto	RA1 a RA5 separados por coma; No aplica justificado	Autor vincula; Steven valida cierre D47.
Severidad, selección	Crítica, Alta, Media o Baja	Caleb y dueño técnico; solo DEF aplicable.
Inicio y Fin, fecha	Ventana prevista y cambio documentado	Responsable para roadmap; no implica tiempo ejecutado.

Nota. Status conserva los seis estados oficiales. Un campo sin asignar no crea una séptima columna.

Las vistas propuestas son Tablero, agrupada por Status; Tabla por sprint, agrupada por Sprint con horas; Hoja de ruta, con Inicio/Fin y agrupación por período; y Bloqueadas, filtrada por Status Bloqueada. Cada vista muestra TAR, responsable y módulo. Aris crea y guarda las vistas en la interfaz siguiendo docs/projects.md; no se afirma que un archivo YAML de repositorio las cree por sí mismo.

Los workflows nativos pueden cambiar el Status de un ítem PR; una PR fusionada no demuestra la aceptación de su issue (GitHub, s. f.-y). La sincronización propuesta usa scripts

de la base confiable, consulta la autorización actual del actor y del autor y contrasta las referencias con la relación PR–issue aprobada por el equipo. Antes de escribir lee el estado anterior. Nunca hace retroceder Terminada ni resuelve Bloqueada por el texto de una PR. Las reaperturas y la salida de un bloqueo conservan las autorizaciones y la evidencia de la Tabla 13. Las transiciones automáticas se limitan a los casos expresamente definidos en docs/projects.md; cualquier situación ambigua exige gestión manual.

La API de Projects requiere autenticación con permisos del Project; GITHUB_TOKEN no los aporta por sí solo (GitHub, s. f.-f). Steven configura PROJECT_OWNER y PROJECT_NUMBER con valores comprobados y conserva el token de alcance mínimo, con vencimiento, en PROJECT_TOKEN. La automatización permanece inactiva mientras falte esa configuración. Una PR externa no autoriza cambios en issues internos aunque mencione sus números. Las reaperturas, la aceptación añadida después del merge, varios PR para una TAR, cierres sin merge, cambios de base y fallos de sincronización se revisan según docs/projects.md. La actualización manual registra estado anterior, motivo y responsable.

Enlaces entre issue, rama, PR y commit

Steven registra en cada issue el nombre de rama, y el autor incorpora TAR/DEF en rama y commits y Refs #n en la PR. La matriz de evidencia relaciona requisito, CP, RA y revisión. El número #n es ejemplo de sintaxis y se sustituye por el issue real; no se inventan enlaces a incidencias inexistentes. Un mismo CP puede verificar varias TAR, con resultados diferenciados por revisión y fixture.

Closes #n se agrega únicamente a la PR que complete todos los criterios de la incidencia, después de que A confirme evidencia y aplique cierre:con-evidencia al issue y a la PR. GitHub interpreta las palabras de cierre cuando la integración alcanza la rama predeterminada; las ignora en PR dirigidas a otra rama (GitHub, s. f.-o). Una corrección de candidata lleva Refs y se cierra al consolidar main o por cierre manual aceptado. Los commits no incluyen claves de cierre para impedir que un cherry-pick cierre trabajo por accidente.

El merge registra integración técnica; no acredita por sí solo la entrada a En prueba. El Plan exige un app-image ZIP ejecutable con etiqueta canónica y los CP aplicables y su fixture definidos. La persona responsable confirma esos datos antes de actualizar el estado. Si faltan, conserva el estado vigente y comenta el pendiente sin crear una columna. Cuando A acepta los resultados y la trazabilidad, cierra la TAR manualmente. Si varias PR contribuyen a una TAR, el responsable identifica cuál completa la integración y revisa las restantes. policy valida referencias, el mensaje propuesto y las etiquetas, pero el revisor y A juzgan la evidencia.

Ciclos de vida y transiciones

La tarea usa los seis estados del Plan. El defecto usa las mismas columnas y registra una resolución adicional en comentario: reproducido, corregido, verificado, duplicado, sin reproducción suficiente o rechazado con motivo. No se agrega una columna para cada resolución. En el defecto, Preparada requiere triage; Terminada por corrección exige CP y variante que reprodujeron el fallo y luego pasaron en la revisión corregida. Un duplicado se cierra con enlace y resolución sin contarlo como defecto corregido.

La Tabla 13 asigna condiciones y evidencia de cada transición. El Apéndice A muestra el diagrama del ciclo; los retornos conservan el historial en comentarios y eventos. El reloj de ciclo no se reinicia al bloquear o reabrir. Caleb comprueba cada defecto reabierto y Steven revisa su efecto en el período y la etiqueta objetivo.

Tabla 13

Transiciones de Tareas y Defectos

Transición	Quién y condición	Evidencia exigida
Ingreso a Preparada	Steven y autor; requisito, estimación, dueño y precedencias suficientes	Formulario y fila del Plan; DEF con triage y pasos o reproducción pendiente explícita.
Preparada → En curso	Responsable; dependencias listas y ninguna otra TAR activa	Comentario de inicio, rama, fecha y objetivo de aceptación.

Transición	Quién y condición	Evidencia exigida
En curso → En revisión	Autor; PR lista, rebase y verify declarado con resultado real	PR, SHA, entorno y salida; draft no acredita preparación para revisar.
En revisión → En curso	Revisor solicita cambio o aparece conflicto	Observación identificada, archivo, causa y siguiente acción.
En revisión → En prueba	Responsable confirma app-image ZIP ejecutable con etiqueta canónica, casos aplicables y fixture definido.	Enlace al ZIP y etiqueta; CP, fixture y entorno. El merge solo registra integración; no demuestra esta condición.
En prueba → Terminada	A del entregable; criterios satisfechos	CP, resultado, fecha, SHA, entorno, etiqueta y RA o exclusión justificada; D47.
En prueba → En curso	Caleb o ejecutor detecta fallo; responsable lo toma	DEF enlazado, pasos y evidencia; no sustituir el resultado fallido.
Estado activo → Bloqueada	Autor; dependencia o recurso impide continuar	Estado anterior, causa, dueño externo/interno, siguiente acción y fecha de revisión.
Bloqueada → estado anterior	Autor y dueño del bloqueo; causa resuelta	Comentario con solución y vínculo; Steven revisa si excedió un día.
Terminada → En curso	Caleb confirma regresión; Steven autoriza reapertura	DEF o resultado nuevo, revisión afectada y motivo; conservar aceptación anterior histórica.

Nota. Transiciones del Plan complementadas con evidencia operativa propuesta. La reapertura exige autorización de Steven.

Comentarios, triage y obsolescencia

docs/comentarios.md ofrece textos completos para reproducción, triage y retrospectiva. Cada comentario identifica hechos observados, revisión y responsable; separa esperado y observado. Caleb coordina reproducción; el dueño técnico estima impacto; Steven decide prioridad y asignación. La retrospectiva del período usa métricas reales o Pendiente, una acción con dueño y criterio verificable, sin convertir una propuesta en acuerdo celebrado.

Tras siete días sin actividad, Caleb comenta qué información o decisión falta y menciona al responsable. A los catorce días Steven revisa duplicidad, vigencia y alcance. No existe cierre automático por antigüedad. Must, DEF Crítica/Alta, riesgos activos y decisiones pendientes permanecen abiertos hasta una resolución explícita; las CP pendientes siguen visibles. Un issue descartado se cierra con motivo y vínculo a D44 si altera alcance, sin contarlo como Terminada aceptada. Las fechas son umbrales propuestos, no plazos de garantía externa.

Pull requests y revisión cruzada

El autor abre draft temprano cuando necesita coordinación, declara pendientes y marca Ready for review al completar los requisitos de entrada. La plantilla del Apéndice B exige requisito, cambio observable, riesgo, reversión, SHA, entorno, verificaciones y lista íntegra de la Tabla 15 del Plan. Cada criterio no aplicable se justifica; las casillas de Markdown se dejan sin marcar en la plantilla. Si cambia UI, se adjuntan capturas Claro y Oscuro de acceso, panel, tabla y diálogo afectados con fecha y revisión (Cuadra et al., 2026a; Cuadra et al., 2026f).

El tamaño recomendado es hasta 400 líneas netas no generadas y quince archivos. Una migración o ajuste transversal puede excederlo con motivo, estrategia de revisión y aprobación del revisor antes de declararse lista. No se mezclan normalización de finales de línea y cambio funcional. El autor explica el comportamiento anterior y el resultante y señala FLU/PAN cuando corresponda; la revisión de un FXML debe conservar las recuperaciones del flujo (Cuadra et al., 2026b).

La Tabla 14 deriva las revisiones de la Tabla 4 y mantiene las responsabilidades A de la Tabla 5 del Plan. Cuando el revisor indicado sea también autor, Steven asigna otro integrante competente y registra relevo; nunca se acepta autoaprobación. Las áreas financieras o de concurrencia pueden requerir consulta de otro dueño además de la aprobación mínima técnica. Esa consulta no crea dos personas A para el mismo entregable.

Tabla 14

Mapa de Revisión y Relevo

Área del cambio	Revisor ajeno propuesto	Consulta o relevo del Plan
FXML, temas y pruebas de Caleb	Steven para interfaz y pruebas	Aris cubre recorrido de interfaz; Caleb revisa interacción cuando no es autor.
Concurrencia y cierre de Steven	Aris	Alfredo revisa numeración y unidad transaccional; Steven conserva invariantes cuando no es autor.
Datos y persistencia de Alfredo	Steven para invariantes; Aris revisa cambio de build relacionado	Alfredo revisa SQL y reportes de otros autores; no se aprueba su propia migración.
HTTP y distribución de Aris	Steven para HTTP simulado; Alfredo para reportes/SQL	Caleb revisa interacción/manual; Aris prepara paquete.
Manual, decisiones y configuración	Caleb o Aris según contenido, ajeno al autor	Steven responde por cierre documental y alcance.

Nota. Revisión técnica ajena y responsabilidad A son funciones distintas; el relevo no permite autoaprobación.

El revisor responde en 24 h y trata con Steven las PR abiertas más de 48 h. Una solicitud de cambios identifica el defecto, la aclaración o el cambio de alcance D44; el autor responde con revisión y evidencia. Antes de integrar se comprueban las conversaciones resueltas, la aprobación ajena vigente y los checks del último SHA. El mensaje final del squash debe coincidir con el bloque validado: el control automatizado del bloque y la comprobación humana del diálogo son pasos distintos.

labels.yml etiqueta por nombre de rama y rutas usando solo la rama base, con pull-requests:write. No ejecuta código del PR en pull_request_target. El módulo de negocio se confirma al triage, pues una ruta transversal puede involucrar varios módulos. CODEOWNERS solicita revisión al salir de draft según los archivos, pero el equipo verifica que la persona

asignada sea distinta del autor y pueda revisar el cambio. Auto-merge no sustituye esos controles.

CI y publicación con GitHub Actions

El Apéndice D entrega ci.yml, policy.yml, release.yml, labels.yml, project.yml, dependabot.yml y configuración de notas. Cada workflow declara permisos; los de pruebas leen contenido, el etiquetado solo escribe PR y el job de publicación dispone contents:write. Los eventos de PR sin confianza no reciben secretos del producto. Las acciones se fijan a versiones exactas v4.2.2, v4.7.1 y v4.6.2, sin referencias mayores flotantes. Aris verifica esas referencias y su procedencia al adoptar; una versión exacta puede moverse, por lo que un SHA completo verificado es la siguiente mejora prevista por D44 (GitHub, s. f.-v).

La Tabla 15 describe la propuesta de comprobaciones. Setup Java proporciona Temurin de la familia 21 y caché Maven; el Wrapper selecciona 3.9.16. Se conserva como única orden Maven de aceptación verify, mediante mvnw o mvnw.cmd según plataforma. El parche exacto del JDK y la imagen del runner se registran en cada ejecución; java-version 21 no fija un parche inmutable. La propuesta usa runners windows-2022 y ubuntu-24.04, no afirma que el runner Windows sea Windows 11 (GitHub, s. f.-h).

Tabla 15

Jobs y Condiciones de Aceptación de CI

Job o control	Propuesta de ejecución	Resultado exigido
windows	Windows, Temurin 21, caché; mvnw.cmd verify	Pruebas unitarias y cobertura; salida y artefactos. No BD real ni prueba visual manual implícita.
postgres	Ubuntu, servicio PostgreSQL 18.6 aislado, V001..V004; ./mvnw verify	Pruebas unitarias e integración activadas por LATCH_CI_DB; error SQL detiene job.
ci	Job agregador, always, necesita windows y postgres	Ambos success; no acepta cancelled o skipped como aprobación.

Job o control	Propuesta de ejecución	Resultado exigido
policy	PR hacia main o release, nombres, mensajes y cierre	Valida commits reales y bloque Squash message coincidente con el título; módulo y pies reales, sin completar datos ficticios.
release	Tag v* desde main; invoca CI y empaqueta Windows	ZIP, SHA-256 y manifest.json; release en borrador; publicación humana pendiente.
Dependabot	Lunes en America/Managua, Maven y Actions	PR propuesta; Steven asigna TAR y traslada a rama build conforme a política.

Nota. Contrato propuesto de CI y publicación; ninguna fila describe una ejecución realizada por esta revisión.

GitHub requiere Linux para los servicios contenedorizados de Actions, por eso PostgreSQL no se declara como servicio del job Windows (GitHub, s. f.-i). Ubuntu utiliza credenciales ficticias solo para la base efímera latch_ci. V001 a V004 se localizan por prefijo y se exige un archivo por prefijo; psql usa ON_ERROR_STOP. T001 aporta datos sintéticos según el Backend y R001 aporta consultas y oráculos de reportes, sin formar parte de la cadena de migraciones. La CI de infraestructura no sustituye los CP de restauración CP-095 ni de respaldo concurrente CP-051, ni acredita CP-067–CP-076, CP-086 o CP-100 por mencionar sus identificadores.

JaCoCo se integra al ciclo verify y exige al menos 80 % de líneas cubiertas en dominio y aplicación. Además del check por paquete, CoverageGate.java comprueba que ambas áreas aporten líneas y superen la meta agregada por área: la ausencia de clases o informe no se acepta como éxito. Los nombres de paquete dominio y aplicacion son una propuesta que debe concordar con la arquitectura real. Surefire falla si no hay pruebas; Failsafe falla si el job PostgreSQL no encuentra integración. La cobertura no prueba por sí sola exactitud financiera, concurrencia ni los cien CP manuales.

La lectura del checkout encontró un Wrapper configurado para Maven 3.8.5 y un POM mínimo con Compiler 3.13.0. Es una observación del esqueleto, no una nueva línea base. El TRD fija Maven 3.9.16, Compiler 3.14.0 y Surefire 3.5.3, junto con las dependencias oficiales, incluidas Gson, HikariCP, Bouncy Castle y las extensiones de JasperReports. El kit entrega docs/pom-ci-fragment.xml y sus instrucciones como ajustes propuestos sobre el POM canónico del TRD; no es un POM completo ni debe sustituir el de la aplicación. Los plugins adicionales de cobertura, integración y empaquetado se someten a D44. No se modificaron el POM real, las migraciones ni las pruebas de producto. Antes de activar policy como check requerido, sus scripts confiables deben estar integrados en la rama base.

El workflow de release es una plantilla pendiente de integrar la provisión confiable del runtime Java 21 con JavaFX en el runner. Comprueba etiqueta y pertenencia a main, repite verify y llama a packaging/package.ps1. El empaquetado usa el Launcher ni.edu.uam.latch.arranque.Launcher del TRD y el runtime previamente verificado. La versión del POM coincide con la etiqueta sin v; rc se conserva en POM y ZIP, mientras jpackage usa su parte numérica. Aris debe comprobar extracción y arranque en Windows 11 x64. El contrato y sus precondiciones constan en docs/contrato-ci.md; esta revisión no acredita un paquete ejecutado.

Se elige secret scanning nativo con push protection como control de secretos; no se agrega una acción externa de gitleaks al flujo base. GitHub documenta scanning gratuito en repositorios públicos y límites de detección por patrones (GitHub, s. f.-t; GitHub, s. f.-u). Steven verifica su activación y push protection en Security, según los pasos oficiales (GitHub, s. f.-k). No se considera que esos controles detecten cualquier contraseña PostgreSQL o clave de respaldo. La revisión humana de diff, capturas y logs sigue siendo obligatoria.

Dependabot propone cambios de Maven y Actions con límite de tres PR abiertas por ecosistema, calendario semanal y etiquetas de tipo y módulo (GitHub, s. f.-j). No cambia de forma automática las versiones fijadas del producto. Steven crea TAR, evalúa D44 y conserva

la PR del bot como origen; Aris incorpora el diff en una rama build/TAR válida, con mensajes y revisión. Durante la candidata solo se aceptan correcciones necesarias, con regresión del área afectada. Los artefactos CI propuestos caducan a los catorce días; Aris conserva la evidencia aceptada en el paquete de entrega antes de ese vencimiento.

Seguridad, dependencias y cumplimiento

El Apéndice E relaciona .gitignore, .gitattributes, .editorconfig y hooks; el Apéndice D explica los ajustes de construcción propuestos sobre el POM oficial. No se versionan LATCH_FX_API_KEY, credenciales PostgreSQL o de respaldo, claves privadas, respaldos, preferencias personales, logs con credenciales ni datos reales. Los ejemplos de variables usan valores ficticios. Un archivo ignorado que ya está rastreado sigue en el historial: su retirada por PR y la revocación de un secreto son acciones distintas.

Los scripts SQL fuente y fixtures sintéticos revisados sí se versionan. El ZIP app-image se conserva como activo de release; respaldos reales y copias bundle con información histórica permanecen en medios restringidos externos al repositorio público. Evidencias de UI recortan o redactan credenciales, rutas personales y datos reales antes de adjuntarse. Caleb comprueba ambos temas y privacidad; Steven revisa contenido sensible y Aris el manifiesto de distribución.

Ante filtración, Steven revoca o rota inmediatamente la credencial y suspende publicación; luego determina alcance, revisión y lugares de exposición. Notifica al equipo por el canal privado académico y al docente si afecta la entrega. El registro público solo conserva identificador y estado redactados. La limpieza posterior usa git filter-repo en un clon nuevo, con respaldo restringido y coordinación de referencias. GitHub advierte que reescribir historia cambia revisiones, puede afectar firmas y exige coordinar clones, forks y referencias (GitHub, s. f.-r).

Después de evaluar el incidente, Steven prepara el reemplazo de referencias, limita temporalmente las reglas solo para la reparación autorizada, registra la excepción D44 y restaura protecciones inmediatamente. Los colaboradores reclonan para evitar reintroducir

objetos. Se revisan PR, Issues, logs y activos afectados; se solicita apoyo de GitHub para referencias que el equipo no pueda limpiar. Se verifica la revocación con el proveedor y se conserva evidencia redactada. Borrar el archivo o reescribir el historial no sustituye esa revocación; no se ejecuta este procedimiento destructivo como parte de la entrega documental.

Las dependencias y plugins se fijan a versión concreta, se revisa el árbol transitivo al adoptar y se registra motivo, licencia, origen y SHA de los artefactos distribuidos. Aris mantiene LICENSES y THIRD-PARTY-NOTICES.txt; Alfredo revisa dependencias persistentes. Una actualización que cambie API, esquema, runtime o licencia se trata como D44 y exige pruebas. Los textos de licencia se obtienen del artefacto o distribución resueltos; el inventario propuesto no acredita que se hayan recopilado ni interpretado sus condiciones.

LICENSE contiene MIT como propuesta para contribuciones propias, pendiente de aceptación de los titulares antes de adoptarse. La visibilidad pública no concede por sí sola permisos equivalentes a una licencia (GitHub, s. f.-n). No se reemplazan por MIT las licencias de JDK, JavaFX, Hibernate, JasperReports ni de sus dependencias. SECURITY.md fija reporte privado y triage propuesto de 24 h; CONTRIBUTING.md consolida el procedimiento; CODE_OF_CONDUCT.md establece convivencia y canal de atención. Steven define el contacto privado si no se habilita el reporte de vulnerabilidades; no se inventa una dirección de correo.

Escalabilidad y mantenimiento

Con cuatro a doce personas se conserva un issue y una rama por objetivo verificable, una TAR En curso por autor y una aprobación técnica ajena. Cada incorporación recibe módulo principal, revisor de relevo, instrucciones de construcción y práctica de recuperación. Steven actualiza CODEOWNERS por paquete con al menos dos personas capaces de revisar el área; Caleb comprueba que los cambios de archivos disparen solicitudes correctas. Si la cola supera 48 h, se prioriza revisar y dividir trabajo antes de iniciar más TAR.

No se crean ramas permanentes de módulos, personas, ambientes o sprint. Una función que no cabe en cinco días se divide por comportamiento y puede integrarse desactivada mediante feature flag, siempre que el camino predeterminado permanezca aceptado y probado. El autor registra flag, dueño, valor inicial y fecha de retiro en la TAR; Steven revisa al cierre de período y exige su limpieza después de aceptar la función. Una flag no permite omitir guardas de seguridad ni publicar pantallas que simulen funcionalidad terminada.

Se mantiene un monorepositorio mientras código, migraciones y distribución necesiten la misma revisión y compatibilidad. Separar repositorios requiere D44 con motivo verificable: ciclo de entrega independiente, contrato estable y responsables suficientes. Cada repositorio futuro conservaría main corta, CI y convenciones; la matriz debe agregar repositorio y SHA para requisitos compartidos. No se divide durante la candidata para resolver conflictos de propiedad o permisos.

Aris adopta límite interno propuesto de 10 MiB por archivo ordinario y revisión especial para evidencias mayores de 5 MiB. Los paquetes se publican en Releases. GitHub avisa al superar 50 MiB y bloquea archivos mayores de 100 MiB en Git ordinario; esos límites de plataforma no son metas del proyecto (GitHub, s. f.-b). Git LFS solo se incorpora con un archivo binario fuente necesario, frecuencia de cambio, costo/cuota y alternativa documentados por D44; no se usa para respaldos, secretos o ZIP regenerables.

Al cierre del sprint Aris revisa ramas ya integradas, conserva SHA y PR antes de borrarlas, ejecuta fetch --prune en sus clones y verifica copia bundle externa. git gc es mantenimiento local después del respaldo, sin --prune=now durante recuperación. Chacon y Straub describen que recuperación depende de objetos aún accesibles; el reflog local no constituye un respaldo remoto (Chacon & Straub, 2014, sección 10.7). Tras HIT-10 se archiva la documentación y se evalúa archivar el remoto con Steven solo cuando estén identificados

pendientes y acceso de entrega; no se ejecuta archivado como efecto automático del documento.

Recuperación y operación diaria

Cada operación comienza con status y una copia del trabajo que no deba perderse. Las situaciones y riesgos completos se incluyen en la tabla del Apéndice G y docs/comandos.md. Para un commit no publicado se usa reset --soft; para uno ya integrado se abre PR con revert. Revert de merge requiere escoger padre con -m y revisar el diff; main lineal normalmente revierte un squash. Ninguna de estas operaciones revierte automáticamente una migración aplicada o un pago confirmado.

El rescate de rama usa reflog del clon que aún conserve objetos y branch rescate SHA antes de gc. Cherry-pick -x traslada una corrección conservando procedencia, con repetición de pruebas y control de duplicados. Stash push -u incluye no rastreados y puede capturar secretos: el autor inspecciona lo que guarda, usa apply para conservar la copia y elimina stash solo tras comprobar el resultado. Los worktrees permiten mantener otra rama en un directorio sin alterar la edición actual; cada entorno usa datos sintéticos y configuración local distinta (Git, s. f.-b).

Para un defecto reproducible se inicia bisect entre una revisión buena y una mala, se ejecuta el mismo CP y fixture en cada paso y se conservan resultados good/bad; bisect reset termina la sesión. Si la prueba es intermitente se estabiliza antes de atribuir una revisión culpable. Git documenta el procedimiento de búsqueda binaria, que requiere una clasificación reproducible (Chacon & Straub, 2014, sección 7.10).

La rutina diaria propuesta comienza cuando cada integrante revisa bloqueos y comentarios, actualiza referencias con fetch, avanza main local con --ff-only y toma una TAR preparada. Registra inicio y módulo, reserva FXML o SQL si aplica y trabaja en rama propia. Antes de publicar revisa diff, privacidad y mensaje, ejecuta verify y usa PR draft o lista según evidencia. Al terminar la jornada deja SHA remoto, estado real, horas realmente registradas y

siguiente acción. Steven revisa bloqueos de más de un día y la cola de PR; Aris comprueba que no queden ramas vencidas sin resolución.

La rutina de cierre de sprint comienza con Caleb consolidando CP y defectos y Alfredo validando migraciones, invariantes y restauración aplicables. Aris reconstruye el paquete aceptable, registra manifiesto y SHA-256 y conserva copias. Steven compara Must, TAR, RA y riesgos y decide si se puede etiquetar; si falta evidencia, registra pendiente y no presenta el hito como satisfecho. Se limpian ramas integradas, se revisan accesos y se registra retrospectiva con métricas observadas. Ninguna rutina exige registrar horas estimadas como ejecutadas.

Integración con el Plan de implementación

La Tabla 16 relaciona los seis períodos con los rangos TAR, ramas ilustrativas, versión e hitos. Una rama ilustrativa define sintaxis, sin afirmar que exista o que el trabajo se ejecutó. Las fechas, los identificadores y la persona A no se renumeran. La matriz por TAR del kit conserva las 96 filas con responsable nominal, requisito y horas estimadas del Plan; esos datos no acreditan trabajo realizado (Cuadra et al., 2026f; Cuadra et al., 2026c).

Tabla 16

Correspondencia de Períodos y Trazabilidad

Período y TAR	Rama de referencia	Etiqueta e HIT	Evidencia prevista
Sprint 0; TAR-001 a TAR-016	feature/TAR-001-base	v0.1.0; HIT-01	JDBC, dos temas, Task, contador y H6.
Sprint 1; TAR-017 a TAR-032	feature/TAR-017-servicios	v0.2.0; HIT-02	Contrato, garantía, ORM y rollback.
Sprint 2; TAR-033 a TAR-048	feature/TAR-038-pago	v0.3.0; HIT-03 y HIT-04	Laboratorio ORM; pago, numeración y anulación.

Período y TAR	Rama de referencia	Etiqueta e HIT	Evidencia prevista
Sprint 3; TAR-049 a TAR-064	feature/TAR-060-reportes	v0.4.0; HIT-05	Diez reportes, consulta oráculo y respaldo.
Sprint 4; TAR-065 a TAR-080	release/v0.9.0-rc.1	v0.9.0-rc.1; HIT-06 y HIT-07	HTTP S1 identificado, regresión y ZIP.
Cierre; TAR-081 a TAR-096	fix/DEF-nnn-observacion, si existe	v1.0.0; HIT-08 a HIT-10	Observaciones, ensayo, RA y copias verificadas.

Nota. Rangos y compromisos del Plan. Las ramas son propuestas de sintaxis; DEF-nnn es metanotación y no un nombre aceptado por la regex.

D44 se registra con formulario de alcance o decisión: solicitante, motivo, alternativa, TAR/RF/RN afectados, impacto en horas estimadas, fecha, riesgo, responsable y resolución de Steven consultando A del área. El cambio no renombra una TAR ya referenciada ni amplía el catálogo CP por conveniencia. Cuando implica esquema, se vincula a Alfredo y al hash de migración; cuando modifica versión o flujo se actualiza ADR-15 y el manifiesto compatible.

D45 se cumple con revisor identificado y ajeno, comentarios numerados como defecto, aclaración o cambio de alcance y una resolución vinculada a SHA. La tabla de relevo del Plan se conserva: Aris cubre coordinación y cierre de Steven; Steven cubre invariantes y sección crítica de Alfredo, quien mantiene responsabilidad especializada por transacción y numeración; Aris cubre recorrido de interfaz de Caleb; Alfredo cubre reportes de Aris y Steven HTTP simulado. No se trasladan credenciales personales con el relevo.

D47 exige relacionar requisito, TAR, CP y variante, resultado, revisión y RA antes de declarar Terminada. La Tabla 17 resume dueños de evidencia; la Tabla 18 reproduce la matriz RACI de entregables del Plan para conservar una sola A. El docente José A. Durán García revisa la evidencia académica, sin asumir aprobación interna de PR ni disponibilidad institucional no confirmada.

Tabla 17

Evidencia RA y Responsables Nominales

RA	Componentes y referencias del Plan	Quién conserva y comprueba
RA1	Controles/eventos JavaFX; TAR-007, 019, 011, 035, 039, 055; CP-077, 079, 080, 087, 088, 092	Caleb conserva capturas y recorrido; Steven revisa UI; A conforme a RACI.
RA2	DataSource/pgJDBC, PreparedStatement, restricciones y rollback; TAR-006, 009, 010, 038, 055	Alfredo conserva SQL y resultados; Steven revisa invariantes.
RA3	H1..H6, dos JVM, cancelación y cierre; TAR-005, 037, 038, 045, 057, 065	Steven conserva logs; Aris revisa concurrencia; Alfredo verifica numeración.
RA4	Hibernate, reportes y H5; TAR-018, 022, 034, 052, 056, 060	Alfredo conserva ORM; Aris reportes; revisión de oráculos independiente.
RA5	HttpClient/Gson y referencia visible; TAR-067, 068, 069; CP-053 a CP-066	Aris conserva evidencia de S1 identificado y cabeceras redactadas; Steven revisa.

Nota. Derivado de la Tabla 27 del Plan. Se enumeran referencias de trazabilidad, no resultados de ejecución.

Tabla 18

Matriz RACI Conservada del Plan

Entregable	Steven Cuadra	Alfredo Montalvan	Caleb Tardencilla	Aris Gavarrete
Alcance y calendario	A/R	C	C	C
Seguridad y roles	A/R	C	C	R
Esquema y migraciones	C	A/R	I	C
Persistencia y transacciones	R	A/R	C	C
FXML y temas	C	I	A/R	R

Entregable	Steven Cuadra	Alfredo Montalvan	Caleb Tardencilla	Aris Gavarrete
Concurrencia y cierre	A/R	R	C	C
HTTP y caché	R	C	C	A/R
Reportes y oráculos	C	R	C	A/R
Pruebas y conformidad visual	R	R	A/R	R
Manual y defensa	A/R	R	R	R
App-image ZIP y respaldo de entrega	C	R	C	A/R

Nota. Asignaciones del Plan, Tabla 5; la guía no cambia responsables ni agrega una segunda A.

Adopción en el calendario real

La propuesta original de adopción parte del 26 de octubre de 2026 y reconoce el cierre de Sprint 0 el 27 de octubre. Se conserva ese calendario sin atribuir trabajo retroactivo ni dar por realizadas las tareas. Los seis oficiales proporcionados llevan fecha de portada 26 de noviembre de 2026; se usan como versiones de contraste entregadas, sin afirmar que se publicaron o consultaron antes. Primero se confirma la decisión de flujo y la configuración; después se ensayan los controles con la base real. La Tabla 19 conserva acciones futuras de adopción y no condiciona la terminación editorial de esta guía a ejecutar la aplicación.

Tabla 19

Plan de Adopción Propuesto

Fecha y estado	Quién y acción	Criterio de listo y verificación
26/10/2026; Propuesto	Steven confirma repositorio personal, cuentas, privacidad y decisión ADR-15; revisa accesos	URL y owner coinciden; invitaciones verificadas; resolución D44 registrada, sin compartir cuenta.

Fecha y estado	Quién y acción	Criterio de listo y verificación
26/10/2026; Propuesto	Cada integrante configura Git, firma y hooks; Aris compara kit con checkout	Versión observada, commit válido e inválido probados; diff del kit revisable; sin reemplazo automático de trabajo.
26/10 a 27/10/2026; Propuesto	Aris fija Wrapper 3.9.16; Alfredo incorpora migraciones auténticas; Caleb pruebas de base	verify con resultado real; fallos documentados. Ausencia de V001..V004 o dominio bloquea aceptar CI.
27/10/2026; Propuesto	Steven y Aris integran bootstrap de scripts y ejecutan primera CI antes de exigir checks	ci y policy aparecen con nombre confirmado; SHA y entorno conservados; no se marca verde si falla.
27/10/2026; Propuesto	Steven importa reglas main/tags; Aris crea etiquetas y milestones; Caleb comprueba formularios	PR de ensayo exige revisión y checks; push prohibido se rechaza; contraste real revisado.
28/10 a 03/11/2026; Propuesto	Aris crea Project; Caleb sincroniza datos y comprueba automatizaciones	Apertura, merge con evidencia y merge sin evidencia producen estados esperados en casos de ensayo.
04/11 a 17/11/2026; Propuesto	Aris ensaya release app-image; Alfredo/Caleb validan esquema y resultados	Borrador con ZIP/hash/manifiesto; extracción y arranque documentados; completar licencias antes de publicar.
18/11 a 25/11/2026; Propuesto	Steven limita cambios de candidata; equipo ejecuta regresión	v0.9.0-rc.1 solo si cumple HIT-07; pendientes y riesgos visibles.
26/11 a 01/12/2026; Propuesto	Equipo atiende observaciones y Steven acepta D47	TAR-092 consolida; HIT-10 con evidencia real, copias y acceso comprobados.

Nota. Acciones de adopción propuestas, sujetas a decisiones y resultados reales; las fechas originales se conservan.

Si las brechas del esqueleto impiden cumplir el primer CI el 27, Steven registra RSK-05 y un ajuste de adopción D44, conserva protección de PR y revisión y deja el check pendiente de habilitar hasta que exista una ejecución interpretable. No se configura un check fantasma

para aparentar control ni se rebaja cobertura silenciosamente. El incumplimiento del criterio se declara en el hito; la reserva y los recortes siguen la política del Plan.

Riesgos del flujo y contingencias

La Tabla 20 vincula escenarios del flujo con los RSK existentes. La probabilidad y el impacto de cada escenario son estimaciones propuestas; no sustituyen los valores ni los dueños del registro canónico del Plan. El nivel se expresa con la escala oficial: verde/bajo hasta 7, ámbar/medio de 8 a 14 y rojo/alto desde 15. Steven revisa los disparadores con los dueños correspondientes. RSK-18 conserva a Steven como dueño; Aris puede ejecutar la conservación de artefactos como una respuesta a la falta de evidencia RA.

Tabla 20

Riesgos del Flujo Vinculados al Registro

RSK, dueño y P × I	Disparador	Mitigación y contingencia
RSK-09; Caleb; 3 × 3 = 9; ámbar/medio	Conflicto FXML o número de migración duplicado	Reserva temporal, ramas breves y revisión del dueño. Resolver con ambos autores y repetir CP; no aceptar solo eliminación de marcadores.
RSK-06; Aris; 2 × 5 = 10; ámbar/medio	Disco falla o remoto inaccesible	Push diario y bundle externo restringido. Recuperar etiqueta/copia verificada; documentar pérdida sin sobrescribir otra copia.
RSK-14; Steven; 2 × 5 = 10; ámbar/medio	Secreto en diff, log, captura o historial	Push protection, revisión y redacción. Revocar, notificar de forma privada y limpiar coordinadamente.
RSK-05; Steven; 4 × 5 = 20; rojo/alto	PR >48 h, rama >5 días o reserva insuficiente	Una TAR activa; revisión primero y división del cambio. Replanificar por D44 y recortar opcional conforme al Plan.
RSK-07; Steven; 3 × 4 = 12; ámbar/medio	Dos jornadas sin persona clave	Relevo y dos dueños de código por área. Redistribuir solo capacidad libre; no compartir cuentas ni aprobar trabajo propio.

RSK, dueño y P × I	Disparador	Mitigación y contingencia
RSK-18; Steven; 2 × 5 = 10; ámbar/medio	Issue cerrado por merge sin CP/RA	Closes condicionado y A revisa evidencia. Reabrir con motivo, corregir Status y completar trazabilidad; no fabricar resultados.
RSK-17; Aris; 3 × 4 = 12; ámbar/medio	CI pasa pero ZIP no arranca o hashes difieren	Ensayo Windows 11 limpio y manifiesto. Bloquear publicación, usar última copia verificada y abrir DEF.
RSK-01/02; Alfredo; 3 × 5 = 15; rojo/alto	RSK-01: motor PostgreSQL no disponible. RSK-02: dialecto o mapeo Hibernate incompatible.	Alfredo distingue motor, mapeo y migración; detener el cambio afectado y aplicar D44 si corresponde. No atribuir todo fallo SQL a RSK-02.
RSK-08; Steven; 3 × 4 = 12; ámbar/medio	Cambio de reglas, alcance o versiones sin acuerdo	Formulario D44 y revisión cruzada. Restaurar configuración aceptada y evaluar impacto antes de continuar.
RSK-18; Steven; 2 × 5 = 10; ámbar/medio	Artefacto CI expira antes de cierre	Aris conserva la evidencia aceptada antes de 14 días; Steven responde por RSK-18. Si expira, repetir con entorno identificado o declarar la falta de evidencia.

Nota. Escenarios propuestos vinculados a riesgos existentes; dueños y escala se conservan del Plan.

Lista de verificación de entrega

La Tabla 21 conserva controles futuros de adopción sin casillas marcadas. Cada responsable registrará fecha, revisión, entorno y enlace al ejecutarlos. La corrección editorial de la guía, la adopción del flujo por el equipo y la comprobación de configuración o producto son estados distintos. El Registro de correcciones y el reporte del kit describen las verificaciones documentales y locales realizadas; ninguna casilla de producto se marca por esa revisión.

Tabla 21

Lista de Entrega y Resultado Esperado

Control sin marcar	Responsable	Resultado esperado
[ ] Documento APA y apéndices	Steven; revisión ajena Caleb	DOCX con portada, estilos APA, PAGE automático, tablas nativas, navegación, referencias y apéndices; kit de texto accesible.
[ ] Fuentes y contraste	Caleb	Fuentes abiertas, consulta real, citas/lista coherentes; colores calculados ≥4,5:1 y revisión visual real pendiente identificada.
[ ] Privacidad e identidad	Steven y cada integrante	Sin secretos ni datos reales; firma verificada y cuentas propias.
[ ] Formularios, etiquetas y milestones	Aris y Caleb	Siete formularios operativos, 47 etiquetas y 16 milestones; fechas del Plan.
[ ] Rulesets y revisión	Steven	PR obligatoria, aprobación ajena, conversaciones resueltas, ci/policy y métodos por destino comprobados.
[ ] CI y cobertura	Aris, Alfredo y Caleb	verify real en Windows/Ubuntu, PostgreSQL aislado, migraciones, pruebas y cobertura de dominio/aplicación ≥80 %.
[ ] Paquete y licencias	Aris	ZIP extraído y ejecutado en Windows 11 x64; SHA-256 coincide; manifiesto y licencias completos.
[ ] Trazabilidad y aceptación	A por entregable; Steven consolida	RF/RN, TAR, CP, DEF, revisión y RA1..RA5 con resultado; excepción explícita si procede.
[ ] Copias y cierre	Aris prepara; Steven comprueba	Etiqueta inmutable, dos copias verificadas y archivos accesibles; horas reales solo si registradas.

Nota. Lista de adopción y comprobación del producto. Las verificaciones de edición constan por separado en el Registro.

Referencias

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/en/v2

Conventional Commits. (s. f.). *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/en/v1.0.0/

Cuadra, S., Montalvan, A., Tardencilla, C., & Gavarrete, A. (2026a). *Brief de diseño de interfaz y experiencia de usuario para Latch* [Documento académico interno no publicado]. Universidad Americana.

Cuadra, S., Montalvan, A., Tardencilla, C., & Gavarrete, A. (2026b). *Documento de flujo de la aplicación de Latch* [Documento académico interno no publicado]. Universidad Americana.

Cuadra, S., Montalvan, A., Tardencilla, C., & Gavarrete, A. (2026c). *Documento de requisitos del producto de Latch* [Documento académico interno no publicado]. Universidad Americana.

Cuadra, S., Montalvan, A., Tardencilla, C., & Gavarrete, A. (2026d). *Documento de requisitos técnicos de Latch* [Documento académico interno no publicado]. Universidad Americana.

Cuadra, S., Montalvan, A., Tardencilla, C., & Gavarrete, A. (2026e). *Esquema de base de datos y servicios de Latch* [Documento académico interno no publicado]. Universidad Americana.

Cuadra, S., Montalvan, A., Tardencilla, C., & Gavarrete, A. (2026f). *Plan de implementación de Latch* [Documento académico interno no publicado]. Universidad Americana.

Driessen, V. (2010, 5 de enero). A successful Git branching model. *nvie.com*. https://nvie.com/posts/a-successful-git-branching-model/

Git. (s. f.-a). *git-rebase*. Recuperado el 26 de octubre de 2026, de https://git-scm.com/docs/git-rebase

Git. (s. f.-b). *git-worktree*. Recuperado el 26 de octubre de 2026, de https://git-scm.com/docs/git-worktree

GitHub. (s. f.-a). *About code owners*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners

GitHub. (s. f.-b). *About large files on GitHub*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github

GitHub. (s. f.-c). *About rulesets*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets

GitHub. (s. f.-d). *About single select fields*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/issues/planning-and-tracking-with-projects/understanding-fields/about-single-select-fields

GitHub. (s. f.-e). *Automatically generated release notes*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/releasing-projects-on-github/automatically-generated-release-notes

GitHub. (s. f.-f). *Automating Projects using Actions*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/issues/planning-and-tracking-with-projects/automating-your-project/automating-projects-using-actions

GitHub. (s. f.-g). *Available rules for rulesets*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets

GitHub. (s. f.-h). *Building and testing Java with Maven*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-maven

GitHub. (s. f.-i). *Creating PostgreSQL service containers*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/actions/tutorials/use-containerized-services/create-postgresql-service-containers

GitHub. (s. f.-j). *Dependabot options reference*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/code-security/reference/supply-chain-security/dependabot-options-reference

GitHub. (s. f.-k). *Enabling push protection for your repository*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/code-security/how-tos/secure-your-secrets/prevent-future-leaks/enable-push-protection

GitHub. (s. f.-l). *GitHub flow*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/get-started/using-github/github-flow

GitHub. (s. f.-m). *GitHub's plans*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/get-started/learning-about-github/githubs-plans

GitHub. (s. f.-n). *Licensing a repository*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/licensing-a-repository

GitHub. (s. f.-o). *Linking a pull request to an issue*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/linking-a-pull-request-to-an-issue

GitHub. (s. f.-p). *Managing auto-merge for pull requests in your repository*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges/managing-auto-merge-for-pull-requests-in-your-repository

GitHub. (s. f.-q). *Permission levels for a personal account repository*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/repositories/managing-your-repositorys-

settings-and-features/repository-access-and-collaboration/permission-levels-for-a-personal-account-repository

GitHub. (s. f.-r). *Removing sensitive data from a repository*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository

GitHub. (s. f.-s). *REST API endpoints for rules*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/rest/repos/rules

GitHub. (s. f.-t). *Secret scanning*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/code-security/concepts/secret-security/secret-scanning

GitHub. (s. f.-u). *Secret scanning detection scope*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/code-security/reference/secret-security/secret-scanning-scope

GitHub. (s. f.-v). *Secure use reference*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/actions/reference/security/secure-use

GitHub. (s. f.-w). *Signing commits*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/authentication/managing-commit-signature-verification/signing-commits

GitHub. (s. f.-x). *Syntax for issue forms*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/syntax-for-issue-forms

GitHub. (s. f.-y). *Using the built-in automations*. Recuperado el 26 de octubre de 2026, de https://docs.github.com/en/issues/planning-and-tracking-with-projects/automating-your-project/using-the-built-in-automations

Preston-Werner, T. (s. f.). *Semantic versioning 2.0.0*. https://semver.org/spec/v2.0.0.html

Apéndice A

Diagramas y leyenda

Las Figuras A1 a A4 representan la propuesta con rótulos y secuencias que conservan su significado sin color. El azul claro identifica integración o entrega y el gris una actividad o condición; el texto y los bordes son negros. Las flechas muestran la secuencia principal y las notas explican las excepciones. Los SVG editables, con título y descripción, se entregan en kit/docs/diagramas. Las ilustraciones no describen ejecuciones observadas.

La Tabla A1 identifica los archivos completos del kit y su uso.

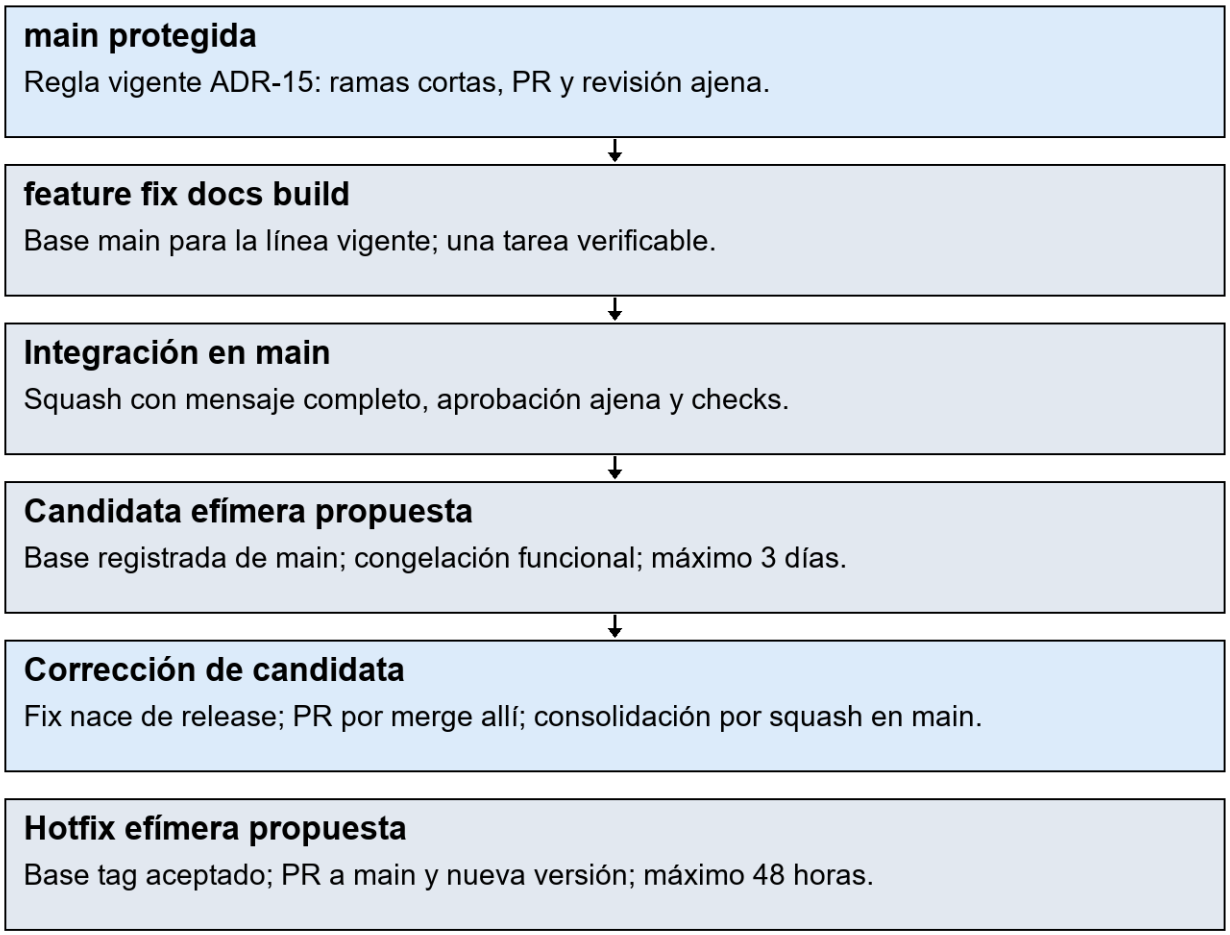
Tabla A1

Archivos auxiliares del apéndice

Archivo del kit	Uso y límite
docs/diagramas/ramas.svg	Fuente editable de la Figura A1.
docs/diagramas/issues.svg	Fuente editable de la Figura A2.
docs/diagramas/ci.svg	Fuente editable de la Figura A3.
docs/diagramas/releases.svg	Fuente editable de la Figura A4.
docs/diagramas/README.md	Leyenda y descripción de las figuras.

Figura A1

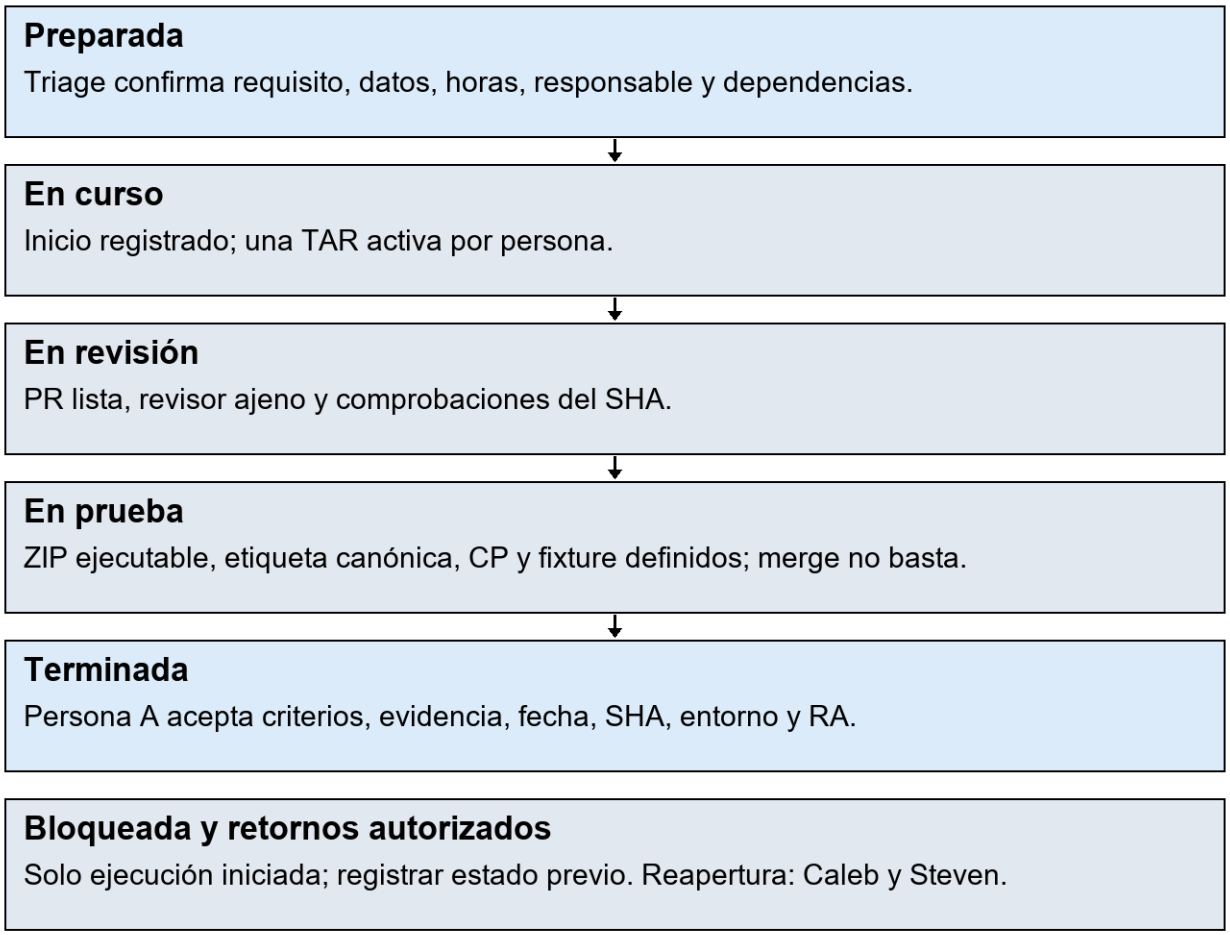
Flujo de ramas



Nota. Main origina ramas de trabajo, integradas por squash. Una candidata puede aislarse en release, recibir correcciones por merge y volver por squash a main antes de etiquetar. Hotfix parte de una etiqueta y vuelve a main mediante PR. Elaboración propia; propuesta del 26/10/2026.

Figura A2

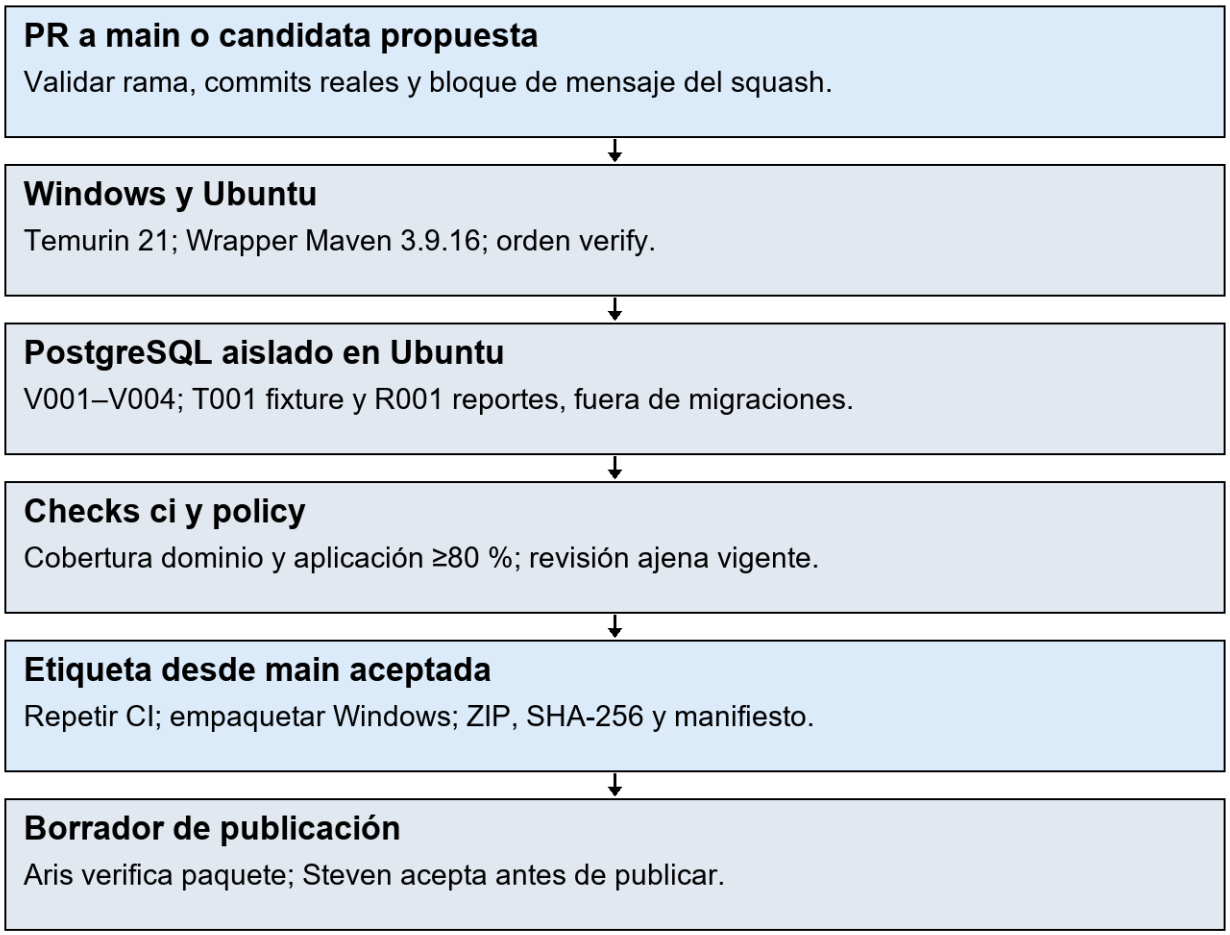
Ciclo de vida de tarea y defecto



Nota. Preparada pasa a En curso, En revisión y En prueba. Terminada exige aceptación con evidencia. Cualquier estado activo puede quedar Bloqueada y regresar tras resolver la causa; una prueba fallida o regresión vuelve a En curso. Elaboración propia; propuesta del 26/10/2026.

Figura A3

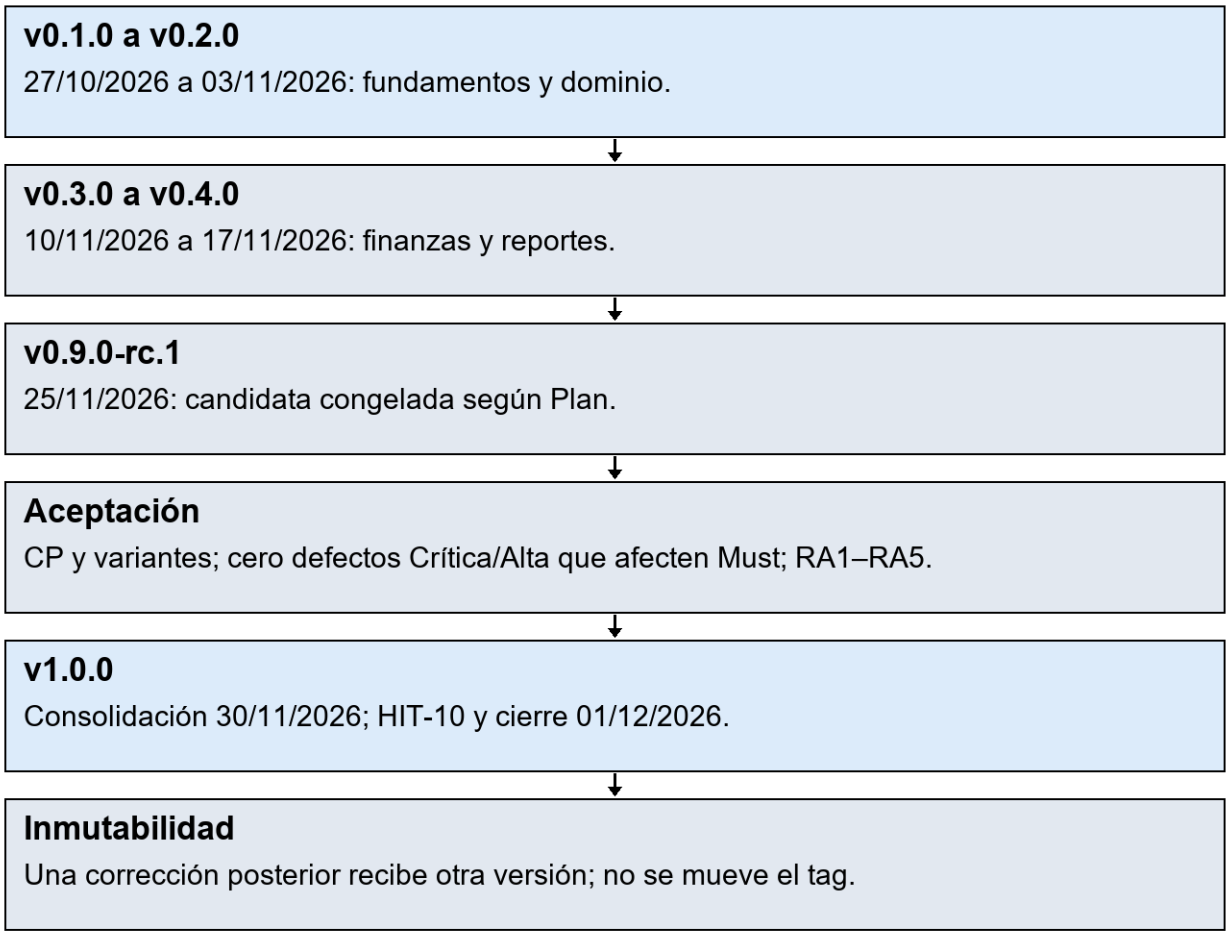
Pipeline de comprobación y publicación



Nota. Cada PR verifica nombres y commits y ejecuta verify en Windows y Ubuntu. Ubuntu agrega PostgreSQL efímero y migraciones. Tras los checks y la aprobación se integra. Una etiqueta inicia nueva CI y empaquetado, calcula huella y crea un borrador sujeto a aceptación. Elaboración propia; propuesta del 26/10/2026.

Figura A4

Modelo de versiones y aceptación



Nota. Las versiones internas avanzan de v0.1.0 a v0.4.0, seguidas por v0.9.0-rc.1 el 25 de noviembre. La aceptación requiere evidencia y defectos Must controlados. Se consolida v1.0.0 para el cierre del 1 de diciembre. Las fechas son metas del Plan. Elaboración propia; propuesta del 26/10/2026.

Apéndice B

Formularios de Issues y plantilla de PR

Se entregan los siete formularios con campos obligatorios y la plantilla de PR. Son archivos de configuración propuestos: la validación YAML no comprueba por sí sola que un requisito exista o que una evidencia sea suficiente. Los archivos no tienen casillas marcadas. Las respuestas se revisan al triage (GitHub, s. f.-x).

La Tabla B1 identifica los archivos completos del kit y su uso.

Tabla B1

Archivos auxiliares del apéndice

Archivo del kit	Uso y límite
`.github/ISSUE_TEMPLATE/01-tarea.yml`	Tarea TAR; completar datos antes de Preparada.
`.github/ISSUE_TEMPLATE/02-defecto.yml`	Reproducción, severidad y vínculo a CP.
`.github/ISSUE_TEMPLATE/03-alcance.yml`	Solicitud D44; no aprueba cambios por sí sola.
`.github/ISSUE_TEMPLATE/04-riesgo.yml`	RSK existente, dueño y escala oficial.
`.github/ISSUE_TEMPLATE/05-decision.yml`	ADR existente, contexto y resolución.
`.github/ISSUE_TEMPLATE/06-prueba.yml`	CP y variante; esperado y observado separados.
`.github/ISSUE_TEMPLATE/07-documentacion.yml`	Archivo, fuente y comprobación documental.
`.github/ISSUE_TEMPLATE/config.yml`	Configuración de apertura de incidencias.
`.github/pull_request_template.md`	Checklist y mensaje de squash completo para validar.
`docs/comentarios.md`	Textos de reproducción, triage y revisión del período.

Apéndice C

Propiedad de código y creación reproducible de registros

CODEOWNERS conserva las cuentas indicadas en la guía; Steven debe confirmar acceso y correspondencia con los paquetes antes de adoptarlo. bootstrap.ps1 usa LitSoloSteven/Latch como destino previsto y solo escribe con -Apply; sin ese parámetro informa la simulación. Los scripts confiables deben estar integrados y los nombres de los checks confirmados antes de activar reglas. Esta revisión solo ensaya datos sintéticos y no ejecuta bootstrap contra GitHub.

La Tabla C1 identifica los archivos completos del kit y su uso.

Tabla C1

Archivos auxiliares del apéndice

Archivo del kit	Uso y límite
CODEOWNERS	Solicitud de revisión por paquetes; confirmar rutas y permisos.
config/labels.json	47 etiquetas; escala de riesgo y nombres independientes del color.
config/milestones.json	16 milestones con fechas del Plan.
scripts/bootstrap.ps1	Simulación por defecto; -Apply escribe solo al adoptarlo.
scripts/create-project.ps1	Creación propuesta de Project y campos; no ejecutada.
scripts/sync-project.ps1	Confianza, relación PR–issue y conservación del estado.
scripts/label-pr.ps1	Etiquetado de PR desde código confiable.
docs/projects.md	Transiciones permitidas y casos de gestión manual.
scripts/project-policy.ps1	Decisión de transición sin escritura remota.

Apéndice D

Workflows y contrato de construcción

Los workflows y scripts de este apéndice se entregan como archivos de texto en el kit. Su sintaxis y los validadores modificados se comprueban localmente; la resolución de dependencias, la ejecución de CI y el ensayo del paquete quedan para adopción. Los jobs con permisos de escritura usan código de la base confiable y verifican la confianza de los datos del evento. Las migraciones V001–V004, los fixtures y las pruebas de producto se obtienen de su especificación oficial; no se implementan en esta corrección.

La Tabla D1 identifica los archivos completos del kit y su uso.

Tabla D1

Archivos auxiliares del apéndice

Archivo del kit	Uso y límite
`.github/workflows/ci.yml`	verify en Windows y Ubuntu con PostgreSQL aislado.
`.github/workflows/labels.yml`	Etiquetado con permisos limitados.
`.github/workflows/policy.yml`	Validación de commits y mensaje propuesto de squash.
`.github/workflows/project.yml`	Sincronización condicionada a confianza y configuración.
`.github/workflows/release.yml`	CI y borrador de distribución, sujetos a adopción.
`.github/dependabot.yml`	Propuestas semanales, sin actualizar la línea base por sí solas.
`.github/release.yml`	Agrupación de notas de publicación.
`packaging/package.ps1`	Empaquetado propuesto; exige ensayo real en Windows 11.
`docs/pom-ci-fragment.xml`	Fragmento de CI sobre POM canónico; nunca reemplazar el POM real.
`scripts/CoverageGate.java`	Control de informe y áreas; no acredita pruebas ejecutadas.

Archivo del kit	Uso y límite
.mvn/wrapper/maven-wrapper.properties	Maven 3.9.16 propuesto, sin JAR bootstrap entregado.
mvnw	Script de texto Unix del Wrapper.
mvnw.cmd	Script de texto Windows del Wrapper.
scripts/validate-pr.py	Título y bloque de squash con los datos reales.
docs/contrato-ci.md	Precondiciones de POM, runtime y empaquetado.

El kit entrega mvnw y mvnw.cmd cotejados con el checkout local y la configuración propuesta de Maven 3.9.16. En .mvn/wrapper se localizó un JAR cuyos metadatos declaran 3.1.0, aunque wrapperUrl apunta a 3.3.2. Se comprobó el contenedor sin ejecutarlo; la huella observada consta en el Registro. Esto no acredita procedencia externa ni pertenencia al kit anterior. No se redistribuye el binario. Antes de adoptar, Aris debe resolver esa diferencia de bootstrap y verificar su origen y la distribución Maven. El manifiesto de esta edición cubre solo los archivos entregados.

Apéndice E

Configuración local hooks y políticas del repositorio

Cada clon instala los hooks mediante scripts/install-hooks.ps1. La instalación local no viaja activa en Git y no sustituye rulesets de main. Los textos de seguridad y convivencia son propuestas del equipo. LICENSE propone MIT para contribuciones propias; no afirma aceptación de titulares ni cambia licencias de terceros. Los textos de terceros se completan desde artefactos resueltos antes de publicar.

La Tabla E1 identifica los archivos completos del kit y su uso.

Tabla E1

Archivos auxiliares del apéndice

Archivo del kit	Uso y límite
`.gitignore`	Archivos generados, datos personales y secretos.
`.gitattributes`	Finales de línea y atributos por tipo de archivo.
`.editorconfig`	Formato de edición compartido.
`config/branch-regex.txt`	Patrón íntegro de nombres de rama.
`scripts/commit-msg.sh`	Validador compartido de mensajes reales.
`scripts/check-policy.sh`	Validación de rango de commits y rama.
`.githooks/commit-msg`	Entrada del hook de mensajes.
`.githooks/pre-push`	Protección local y excepciones explícitas de candidata.
`scripts/install-hooks.ps1`	Instalación por clon, sin cambios globales.
`docs/instalacion-git.md`	Identidad, credenciales, firma y comprobación local.
`SECURITY.md`	Reporte privado; el contacto se confirma al adoptar.
`CONTRIBUTING.md`	Contribución y revisión ajena.

Archivo del kit	Uso y límite
CODE_OF_CONDUCT.md	Convivencia y tratamiento de incidentes.
LICENSE	MIT propuesta para código propio; aceptación pendiente.
LICENSES/README.md	Obtención de textos de terceros desde artefactos resueltos.
THIRD-PARTY-NOTICES.txt	Inventario propuesto, pendiente de completar con artefactos.
docs/evidencia/README.md	Trazabilidad, privacidad y conservación.
docs/adr/ADR-15-enmienda-git.md	Enmienda propuesta; regla vigente y aceptación separadas.
scripts/check-release-ref.sh	Autoriza solo creación o retirada específica de candidata.
docs/release-operacion.md	Base de correcciones y operación compatible con el hook.

Apéndice F

Rulesets exportables y pasos en la interfaz

Los JSON siguientes son cargas de creación/exportación propuestas, con condiciones por referencia y reglas conforme a la API oficial (GitHub, s. f.-s). No se agregan IDs ficticios de repositorio o ejecución. RepositoryRole 5 identifica administración para la excepción de creación de tags; Steven confirma el selector al importar. La regla de inmutabilidad separada carece de bypass. main bloquea borrado; release admite limpieza posterior por Aris como excepción declarada. Los JSON no acreditan aceptación de la API.

La Tabla F1 identifica los archivos completos del kit y su uso.

Tabla F1

Archivos auxiliares del apéndice

Archivo del kit	Uso y límite
`config/rulesets/main.json`	Main: revisión ajena, squash y linealidad.
`config/rulesets/release.json`	Candidata efímera: merge revisado y limpieza autorizada.
`config/rulesets/tags-create.json`	Creación de etiquetas por administración.
`config/rulesets/tags-immutable.json`	Inmutabilidad de etiquetas sin bypass.
`docs/reglas-web.md`	Pasos de General y reglas coherentes con el squash.

Apéndice G

Glosario y tabla de comandos

La Tabla G1 define términos usados en la estrategia. La Tabla G2 reúne comando, situación y riesgo. Las secuencias se leen paso a paso; ningún comando de reescritura se ejecuta como parte de la revisión documental. SHA, ruta, número y etiqueta se sustituyen por valores comprobados antes de operar.

`config/glosario.json`
`config/comandos.json`
`docs/comandos.md`

Tabla G1

Glosario del Flujo

Término	Definición
ADR	Registro de decisión de arquitectura; conservar ADR-01 a ADR-22.
D44 / D45 / D47	Configuración y cambios / revisión cruzada / trazabilidad y cierre del Plan.
TAR / DEF / CP	Tarea / defecto / caso de prueba; el número de issue es independiente.
RA	Resultado de aprendizaje RA1 a RA5; no equivale al estado de un tablero.
RACI	R ejecuta, A responde por cierre, C se consulta, I se informa.
PR	Propuesta de integración con revisión independiente y comprobaciones.
Squash	Condensa una rama en un commit nuevo al integrar.
Rebase	Reaplica cambios y crea SHA nuevos; solo historial propio.
Lease	Condición que evita reemplazar una referencia remota distinta de la conocida.
Candidata rc	Versión preliminar sometida a aceptación; no es v1.0.0.
App-image	Directorio de aplicación con runtime; se distribuye ZIP y se prueba tras extraer.
SHA / SHA-256	Identificador de revisión Git / huella criptográfica de archivo; usos distintos.
Main desplegable	Meta normativa: revisión aceptada y paquete construible; no afirmación de aplicación ejecutada.

Término	Definición
Feature flag	Control para mantener desactivada una función incompleta integrada y probada.
Propietario temporal	Persona que coordina edición conflictiva de archivo durante una ventana registrada.
Status check	Resultado asociado a una revisión; required checks bloquean integración según reglas.

Nota. Definiciones operativas propuestas para el equipo.

Tabla G2

Operación y Recuperación por Situación

Comando	Cuándo usarlo	Riesgo y comprobación
git status --short --branch	Al comenzar y antes de cambiar de rama	Lectura; revisar archivos no rastreados.
git fetch --prune origin	Inicio diario; actualizar referencias	Lectura remota; poda referencias, no ramas locales.
git switch main; git pull --ff-only	Actualizar main local limpia	Rechaza divergencia; no resolver con reset automático.
git switch -c feature/TAR-001-base	Iniciar TAR preparada desde main	Crea rama; nombre ilustrativo, no acredita ejecución de TAR-001.
git rebase origin/main	Antes de PR, solo rama propia	Reescribe SHA; ante duda git rebase --abort.
git push --force-with-lease origin RAMA_PROPIA	Tras rebase de rama propia publicada, avisando al revisor	Reescribe remoto solo si lease coincide; no usar en main/release.
git restore --staged ARCHIVO	Retirar del índice sin perder edición	No elimina el archivo de trabajo.
git reset --soft HEAD~1	Deshacer último commit aún no publicado	Reescribe historial local; conserva contenido en índice.
git revert SHA	Deshacer cambio publicado mediante nuevo PR	Puede generar conflictos; revisar estado financiero/esquema.
git reflog; git branch rescate SHA	Recuperar rama borrada en el mismo clon	Reflog es local y expira; no garantiza rescate tras gc.

Comando	Cuándo usarlo	Riesgo y comprobación
git cherry-pick -x SHA	Trasladar corrección aceptada a candidata o línea vigente	Puede duplicar cambio; -x conserva procedencia; abortar con --abort.
git revert -m 1 SHA_MERGE	Revertir merge de corrección en release/*	Verificar padre 1 con git show; no revierte BD automáticamente.
git bisect start; git bisect bad; git bisect good ETIQUETA	Localizar primer cambio que introduce defecto	Ejecutar CP determinista, marcar good/bad y finalizar git bisect reset.
git stash push -u -m TAR-001-pausa; git stash apply	Pausar trabajo sin secretos para una corrección	-u incluye no rastreados; apply conserva stash; borrar solo tras verificar.
git worktree add ../latch-defecto -b fix/DEF-001-caso origin/main	Revisar defecto sin alterar la TAR actual	Nuevo directorio; no compartir BD operativa ni secretos entre worktrees.
git worktree remove ../latch-defecto	Retirar worktree tras revisar git status	Rechaza cambios; no forzar para eludir trabajo no guardado.
git branch -d RAMA; git push origin --delete RAMA	Rama integrada con evidencia y SHA conservado	En squash -d puede rechazar; Aris verifica contenido antes de borrar con -D.
git bundle create COPIA.bundle --all	Cierre de sprint, destino fuera del repo público	Incluye historia y posibles secretos históricos; acceso restringido.
git fsck --full; git gc	Mantenimiento local por Aris al cierre, tras respaldo	gc elimina objetos según expiración; evitar --prune=now durante rescate.
git log --show-signature -1; git verify-tag v1.0.0	Verificar firmas y etiqueta existentes	Requiere claves de confianza; el nombre de etiqueta no prueba autenticidad.
git filter-repo --sensitive-data-removal --invert-paths --path RUTA_FILTRADA	Incidente coordinado por Steven después de revocar	Solo copia nueva y respaldo restringido; cambia SHA/firmas, exige reclonar.
.\mvnw.cmd verify	Aceptación local Windows y cierre de sprint	Puede fallar por red, versiones, pruebas o cobertura; conservar salida real.

Nota. Los puntos y coma separan pasos; comprobar cada salida antes del siguiente. No usar reset o force sobre historia compartida sin procedimiento autorizado.

Apéndice H

Fuentes y trazabilidad documental

La Tabla H1 relaciona afirmaciones y fuentes. Las fuentes externas se recuperaron para esta revisión el 26 de octubre de 2026 en America/Managua. Los seis oficiales se tratan como seis obras, con PDF y DOCX como representaciones de cada una. Se contrastaron las secciones y tablas pertinentes antes de escoger la representación de lectura; el Registro documenta cobertura y límites. La tabla no acredita consultas históricas al remoto ni resultados de producto.

Tabla H1

Fuentes de Afirmaciones y Límite de Uso

Afirmación	Fuente	Consulta	Sección	Confiabilidad
Bisect localiza un cambio por resultados reproducibles.	Pro Git (Chacon & Straub, 2014, sección 7.10)	26/10/2026	7.10; Binary search	Fuente primaria.
Configuración global y local, identidad y normalización de texto.	Pro Git (Chacon & Straub, 2014, sección 8.1)	26/10/2026	8.1; core.autocrlf; scopes	Fuente primaria.
Procedencia oficial de instalación; versión exacta del encargo es fijación del equipo.	Pro Git (Chacon & Straub, 2014, sección 1.5)	26/10/2026	1.5; Windows	Fuente primaria.
Reflog, recuperación y mantenimiento local; límites tras eliminación de objetos.	Pro Git (Chacon & Straub, 2014, sección 10.7)	26/10/2026	10.7; Data recovery	Fuente primaria.
Tipo, alcance opcional, descripción y pies; adaptación de Latch conserva tipos existentes.	Conventional Commits 1.0.0 (Conventional Commits, s. f.)	26/10/2026	Specification; Examples	Fuente primaria.

Afirmación	Fuente	Consulta	Sección	Confiabilidad
Capturas de Claro/Oscuro y PAN conforme al contrato visual.	Brief de diseño de interfaz y experiencia de usuario para Latch (Cuadra et al., 2026a)	26/10/2026	Temas y pantallas	Obra oficial interna; planificación.
FLU conserva significado de recuperación y operaciones.	Documento de flujo de la aplicación de Latch (Cuadra et al., 2026b)	26/10/2026	FLU y recuperación	Obra oficial interna; planificación.
RF-Mxx-nn, RN-01..38 y prioridades originales.	Documento de requisitos del producto de Latch (Cuadra et al., 2026c)	26/10/2026	Módulos y reglas	Obra oficial interna; planificación.
ADR-15 ya adopta main protegida y ramas cortas; versiones preservadas.	Documento de requisitos técnicos de Latch (Cuadra et al., 2026d)	26/10/2026	ADR-15 y componentes fijados	Obra oficial interna; planificación.
Revisión SQL y compatibilidad de esquema sin inventar scripts.	Esquema de base de datos y servicios de Latch (Cuadra et al., 2026e)	26/10/2026	Migraciones y transacciones	Obra oficial interna; planificación.
Calendario, RACI, TAR/CP, HIT, RSK y D44/D45/D47.	Plan de implementación de Latch (Cuadra et al., 2026f)	26/10/2026	Tablas 4, 5, 10, 15, 21 y 27; Apéndice G	Obra oficial interna; planificación.
Modelo original con dos ramas permanentes; comparación para decisión local.	A successful Git branching model (Driessen, 2010)	26/10/2026	The main branches; Release branches; Hotfix branches	Fuente primaria.
Rebase reescribe revisiones; abort permite volver ante conflicto.	git-rebase (Git, s. f.-a)	26/10/2026	DESCRIPTION ; MODE OPTIONS	Fuente primaria.
Worktrees permiten directorios de trabajo de ramas distintas compartiendo repositorio.	git-worktree (Git, s. f.-b)	26/10/2026	DESCRIPTION ; COMMANDS	Fuente primaria.

Afirmación	Fuente	Consulta	Sección	Confiabilidad
CODEOWNERS exige Write a propietarios; se usa primera ubicación y última coincidencia.	About code owners (GitHub, s. f.-a)	26/10/2026	Who can use this feature?; Syntax; Location	Fuente primaria.
Aviso desde 50 MiB y bloqueo normal sobre 100 MiB; usar releases para paquetes.	About large files on GitHub (GitHub, s. f.-b)	26/10/2026	File size limits; Distributing large binaries	Fuente primaria.
Rulesets de ramas y tags disponibles en Free público; push rulesets no se incluyen.	About rulesets (GitHub, s. f.-c)	26/10/2026	Who can use this feature?; Branch and tag rulesets	Fuente primaria.
Campos de selección y filtros para Status, módulo, sprint y severidad.	About single select fields (GitHub, s. f.-d)	26/10/2026	Adding a single select field	Fuente primaria.
Notas automáticas resumen PR, colaboradores y cambios; requieren revisión humana.	Automatically generated release notes (GitHub, s. f.-e)	26/10/2026	About automatically generated release notes	Fuente primaria.
Automatizar Project requiere autenticación apropiada además de permisos del repositorio.	Automating Projects using Actions (GitHub, s. f.-f)	26/10/2026	GitHub Actions workflows; Authentication	Fuente primaria.
Historial lineal impide merge commits; se requieren métodos compatibles.	Available rules for rulesets (GitHub, s. f.-g)	26/10/2026	Require linear history; Require a pull request; Require status checks	Fuente primaria.
Setup Java y caché Maven en Actions; propuesta adapta verify al Wrapper.	Building and testing Java with Maven (GitHub, s. f.-h)	26/10/2026	Building and testing; Caching; Artifacts	Fuente primaria.

Afirmación	Fuente	Consulta	Sección	Confiabilidad
Los servicios contenedorizados de Actions se ejecutan en Linux; Windows tiene job separado.	Creating PostgreSQL service containers (GitHub, s. f.-i)	26/10/2026	Prerequisites; Running jobs directly on runner	Fuente primaria.
Actualizaciones propuestas de Maven y Actions mediante dependabot.yml.	Dependabot options reference (GitHub, s. f.-j)	26/10/2026	package-ecosystem; schedule; labels; commit-message	Fuente primaria.
El propietario verifica y habilita push protection en Security; no se presume activación.	Enabling push protection for your repository (GitHub, s. f.-k)	26/10/2026	Who can use; Steps	Fuente primaria.
Ramas, PR y revisión como base ligera de colaboración.	GitHub flow (GitHub, s. f.-l)	26/10/2026	Following GitHub flow	Fuente primaria.
Free público como base del flujo; no se presupone Education.	GitHub’s plans (GitHub, s. f.-m)	26/10/2026	GitHub Free for personal accounts	Fuente primaria.
Repositorio público no sustituye licencia; propuesta MIT solo para contribuciones propias.	Licensing a repository (GitHub, s. f.-n)	26/10/2026	Choosing the right license	Fuente primaria.
Closes se interpreta al integrar en rama predeterminada; release no cierra por sí sola.	Linking a pull request to an issue (GitHub, s. f.-o)	26/10/2026	About linked issues; Linking using a keyword	Fuente primaria.
Auto-merge está disponible en Free público. El procedimiento manual de esta guía propone mantenerlo desactivado hasta acordar un mensaje final equivalente.	Managing auto-merge for pull requests in your repository (GitHub, s. f.-p)	26/10/2026	Who can use this feature?	Fuente primaria.

Afirmación	Fuente	Consulta	Sección	Confiabilidad
El repositorio personal tiene propietario y colaboradores; no roles Maintain/Triage separados.	Permission levels for a personal account repository (GitHub, s. f.-q)	26/10/2026	Owner access; Collaborator access	Fuente primaria.
Revocar antes de limpiar historial; coordinar clones, firmas y referencias afectadas.	Removing sensitive data from a repository (GitHub, s. f.-r)	26/10/2026	About removing; Side effects; Purging a file	Fuente primaria.
JSON de rulesets y scripts REST reproducibles; su aceptación remota queda pendiente.	REST API endpoints for rules (GitHub, s. f.-s)	26/10/2026	Create a repository ruleset; parameters; update	Fuente primaria.
Secret scanning automático y gratuito en repositorios públicos; no cubre toda contraseña posible.	Secret scanning (GitHub, s. f.-t)	26/10/2026	How can I access this feature?	Fuente primaria.
Cobertura depende de patrones y límites de análisis; no garantiza ausencia de secretos.	Secret scanning detection scope (GitHub, s. f.-u)	26/10/2026	Push protection limitations	Fuente primaria.
Permisos mínimos y referencias fijadas; no ejecutar código de PR con secretos privilegiados.	Secure use reference (GitHub, s. f.-v)	26/10/2026	Using third-party actions; Script injections	Fuente primaria.
GitHub admite firmas SSH o GPG; se debe verificar resultado y confianza.	Signing commits (GitHub, s. f.-w)	26/10/2026	Signing commits	Fuente primaria.
Formularios en .github/ISSUE_TEMPLATE; respuestas se convierten a Markdown.	Syntax for issue forms (GitHub, s. f.-x)	26/10/2026	Top-level syntax; About YAML syntax	Fuente primaria.

Afirmación	Fuente	Consulta	Sección	Confiabilidad
La automatización nativa de PR actúa sobre ítem PR; no acredita aceptación del issue.	Using the built-in automations (GitHub, s. f.-y)	26/10/2026	Using built-in automations	Fuente primaria.
Destino remoto previsto; sus metadatos y configuración deben confirmarse.	Guía original y lectura de .git/config local.	26/10/2026	Inventario local; sin remoto configurado.	Observación local; sin consulta remota.
MAJOR.MINOR.PATCH, contrato público declarado y candidatas; no mover versiones publicadas.	Semantic versioning 2.0.0 (Preston-Werner, s. f.)	26/10/2026	Specification 1 – 11	Fuente primaria.

Nota. Las referencias completas figuran en la lista alfabética. Las fuentes sustentan capacidades o especificaciones, según corresponda; ninguna fila prueba activación remota ni resultados de producto. Este límite se aplica a toda la tabla.

La Tabla H2 identifica la matriz de 96 TAR del Plan, conservada en kit/docs/trazabilidad-tar.csv con módulos, requisitos, responsables, horas y precedencias. El CSV conserva 228 horas estimadas; no son horas ejecutadas. Pendiente indica que esta revisión documental no atribuye ejecución a las filas, sin reemplazar el estado real que el equipo registre en sus herramientas.

Tabla H2

Índice de la Matriz TAR Conservada

Filas	Fuente y destino	Límite de evidencia
TAR-001 a TAR-096	Plan, tablas A1..A6; docs/trazabilidad-tar.csv	96 registros, 228 horas estimadas; no horas ejecutadas.
RF/RN, CP y RA	Plan, Tabla 27 y catálogo CP; cuerpo, Tablas 13 y 17	Cada cierre añade CP, resultado, SHA y entorno; no asignar resultados desde el CSV.

Nota. La fila de TAR conserva responsables y estimaciones nominales del Plan; las tareas de adopción se imputan mediante D44 sin inventar horas ya realizadas.

La Tabla H3 identifica los archivos completos del kit y su uso.

Tabla H3

Archivos auxiliares del apéndice

Archivo del kit	Uso y límite
`docs/trazabilidad-tar.csv`	96 TAR y 228 horas estimadas, sin renumeración.
`README.md`	Alcance, orden de adopción y archivos efectivamente entregados.
`REPORTE_VERIFICACION.md`	Resultados locales y límites de lo comprobado.
`MANIFEST-SHA256.txt`	Huellas de los archivos entregados en el kit.